

Wydział Elektroniki i Technik Informacyjnych  
Politechnika Warszawska

Przedmiot: Modelowanie i identyfikacja (MODI)

## Sprawozdanie z projektu nr 2

Identyfikacja modeli statycznych i dynamicznych  
metodą najmniejszych kwadratów

Autor: Mikołaj Wróbel (337021)

Warszawa, 2026

# Spis treści

- Założenia i metoda
- 1. Identyfikacja modeli statycznych
  - 1a. Dane statyczne
  - 1b. Statyczny model liniowy
  - 1c. Statyczne modele wielomianowe
- 2. Identyfikacja modeli dynamicznych
  - 2a. Dane dynamiczne
  - 2b. Dynamiczne modele liniowe ARX
  - 2c. Dynamiczne modele nieliniowe NARX
  - 2d. Charakterystyka statyczna z modelu NARX
- 3. Identyfikacja modelem neuronowym (Keras)
  - 3a. Sieć zbyt mała (1 neuron ukryty)
  - 3b. Sieć umiarkowana (10 neuronów ukrytych)
  - 3c. Sieć zbyt duża (500 neuronów ukrytych)
  - 3d. Zestawienie wyników
  - 3e. Omówienie wyników

Sprawozdanie zostało wykonane w środowisku Jupyter Notebook, w związku z czym jest w pełni edytowalnym dokumentem. Komórki kodu uruchamiają zewnętrzne pliki źródłowe w języku Python, obliczają metryki oraz generują odpowiednie wykresy. W przypadku modyfikacji list ze stopniami lub rzędami modeli, wystarczy ponownie uruchomić wszystkie komórki notatnika.

## Założenia i metoda

Modele wyznaczono metodą najmniejszych kwadratów bez użycia specjalizowanych bibliotek identyfikacyjnych. Dla macierzy regresorów  $\Phi$  i wektora wyjść  $y$  parametry modelu spełniają równanie normalne:

$$\Phi^T \Phi \theta = \Phi^T y. \quad (1)$$

W implementacji układ normalny jest rozwiązywany solverem eliminacji Gaussa z częściowym wyborem elementu głównego. Dokładność oceniono metrykami:

$$MSE = \frac{1}{N} \sum_{k=1}^N (y(k) - \hat{y}(k))^2, \quad RMSE = \sqrt{MSE}. \quad (2)$$

### Listing 1. Solver MNK – rozkład QR metodą refleksji Householdera (modele.py)

```
class MNKSolver:
    @staticmethod
    def solve(Phi: np.ndarray, y: np.ndarray) -> np.ndarray:
        R, d = Phi.astype(float).copy(), y.astype(float).copy()
        N, M = R.shape
        # Redukcja do postaci górnotrójkątnej – kolejne refleksje
        Householdera
        for k in range(M):
            x = R[k:, k]
            norm_x = np.linalg.norm(x)
            if norm_x < 1e-12:
                continue
            s = 1.0 if x[0] >= 0 else -1.0
            v = x / (x[0] + s * norm_x); v[0] = 1.0
            beta = 2.0 / np.dot(v, v)
            R[k:, k:] -= beta * np.outer(v, np.dot(v, R[k:, k:]))
            d[k:] -= beta * v * np.dot(v, d[k:])
        # Podstawienie wsteczne
        theta = np.zeros(M)
        for i in range(M - 1, -1, -1):
            if abs(R[i, i]) < 1e-12:
                return np.full(M, np.nan)
            theta[i] = (d[i] - np.dot(R[i, i+1:], theta[i+1:])) / R[i,
i]
        return theta
```

```
In [2]: from pathlib import Path
import html
import importlib.util
import os
import subprocess
import sys

import numpy as np
import matplotlib.pyplot as plt
from IPython.display import HTML, Image, Markdown, display

# Jeśli notebook zostanie otwarty z innego katalogu, znajdź katalog projektu po
def find_project_root(start: Path) -> Path:
    for candidate in [start, *start.parents]:
        if (candidate / "danestat28").exists() and (candidate / "danedynucz28").
            return candidate
    raise FileNotFoundError("Nie znaleziono katalogu projektu z folderami danych")

ROOT = find_project_root(Path.cwd().resolve())
STATIC_CODE = ROOT / "indyfikacja_modeli_statycznych" / "kod"
DYNAMIC_CODE = ROOT / "indyfikacja_modeli_dynamicznych" / "kod"

GENERATE_NOTEBOOK_FIGURES = True
```

## Uruchomienie kodu źródłowego

Ta komórka wywołuje oba skrypty główne: część statyczną i dynamiczną. Skrypty generują wszystkie wykresy używane dalej w sprawozdaniu. W celu przejrzenia raportu bez ponownego przeliczania wyników, należy ustawić zmienną RUN\_MAIN\_SCRIPTS na wartość False.

```
In [4]: RUN_MAIN_SCRIPTS = True

def run_project_script(script: Path):
    env = os.environ.copy()
    env["MPLBACKEND"] = "Agg"
    env["PYTHONIOENCODING"] = "utf-8"
    result = subprocess.run(
        [sys.executable, str(script)],
        cwd=str(ROOT),
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="replace",
        env=env,
    )
    result.check_returncode()
    return result

if RUN_MAIN_SCRIPTS:
    run_project_script(STATIC_CODE / "main.py")
    run_project_script(DYNAMIC_CODE / "main.py")
```

# 1. Identyfikacja modeli statycznych

Dane statyczne podzielono na zbiór uczący i weryfikujący w proporcji 60% do 40%. W pierwszej kolumnie danych znajduje się wejście  $u$ , a w drugiej wyjście  $y$ .

```
In [5]: static_import = load_module("modi_static_import", STATIC_CODE / "import_danych.p
static_models = load_module("modi_static_models", STATIC_CODE / "modele.py")
static_plotting = load_module("modi_static_plotting", STATIC_CODE / "rysowanie.p

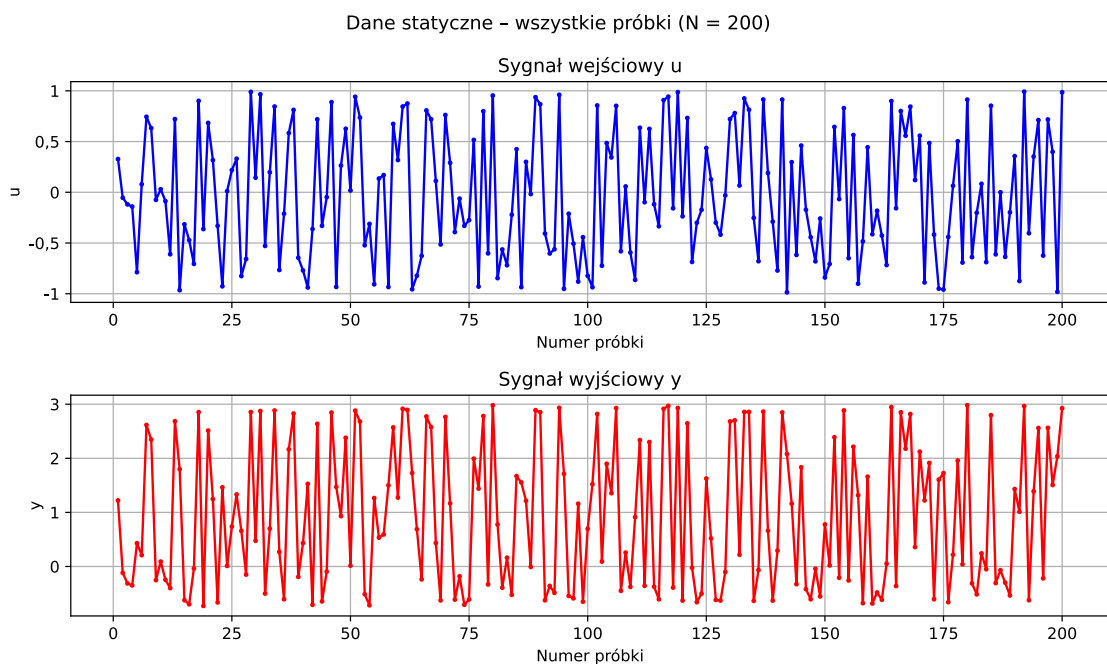
STATIC_SPLIT = 0.6
STATIC_POLY_DEGREES = list(range(1, 21))

dane_stat = static_import.Import_Static_Data(split=STATIC_SPLIT)
plotter_stat = static_plotting.PlotData(dane_stat.out_dir)

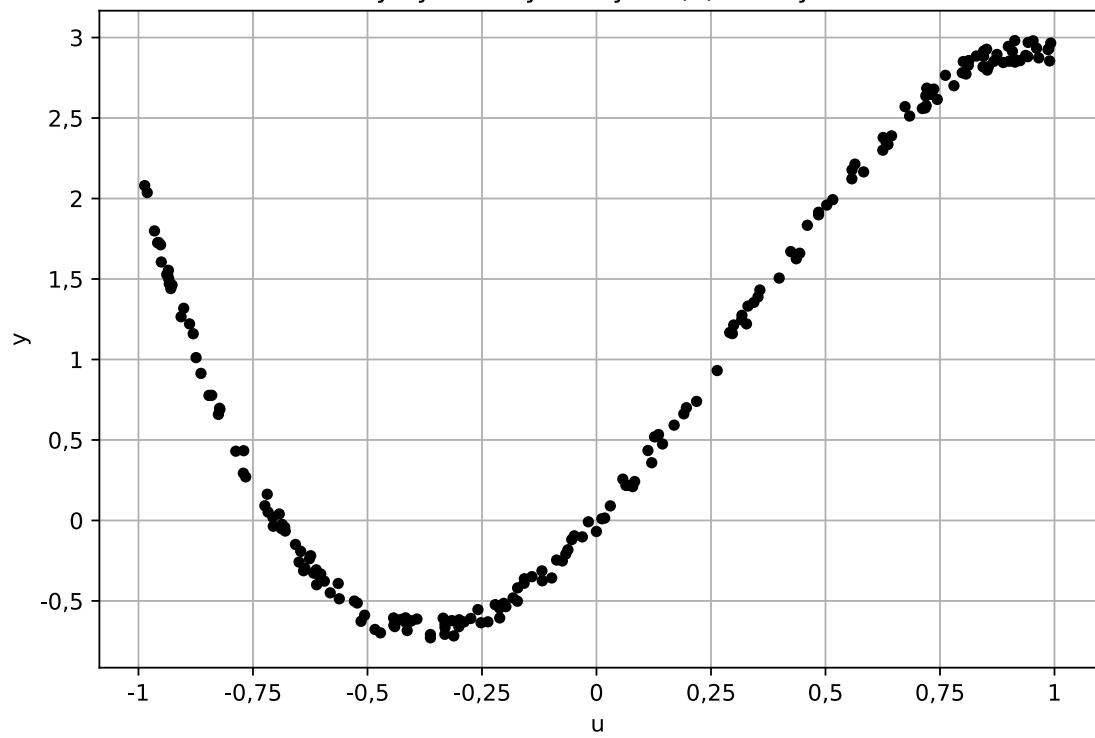
if GENERATE_NOTEBOOK_FIGURES:
    plotter_stat.plot_all_data(dane_stat.u, dane_stat.y)
    plotter_stat.plot_static_characteristic(dane_stat.u, dane_stat.y)
    plotter_stat.plot_training_data(dane_stat.u_ucz, dane_stat.y_ucz)
    plotter_stat.plot_training_characteristic(dane_stat.u_ucz, dane_stat.y_ucz)
    plotter_stat.plot_verification_data(dane_stat.u_wer, dane_stat.y_wer, dane_s
    plotter_stat.plot_verification_characteristic(dane_stat.u_wer, dane_stat.y_w
    plt.close("all")
```

## 1a. Dane statyczne

Na rysunkach 1 i 2 pokazano przebiegi oraz charakterystyki statyczne dla wszystkich danych, zbioru uczącego i zbioru weryfikującego.

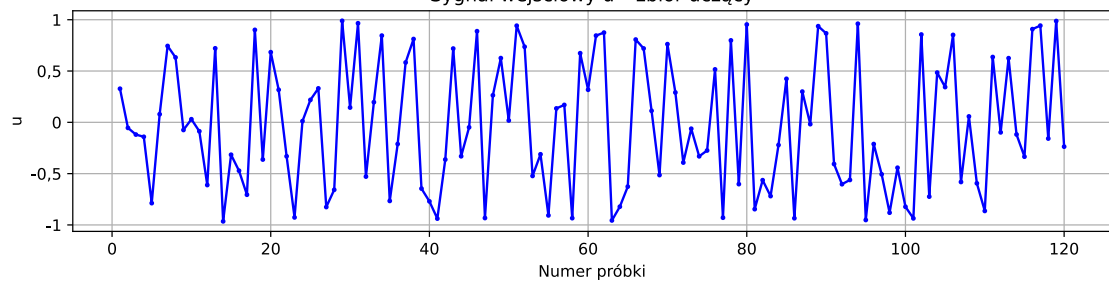


Charakterystyka statyczna  $y = f(u)$  - wszystkie dane

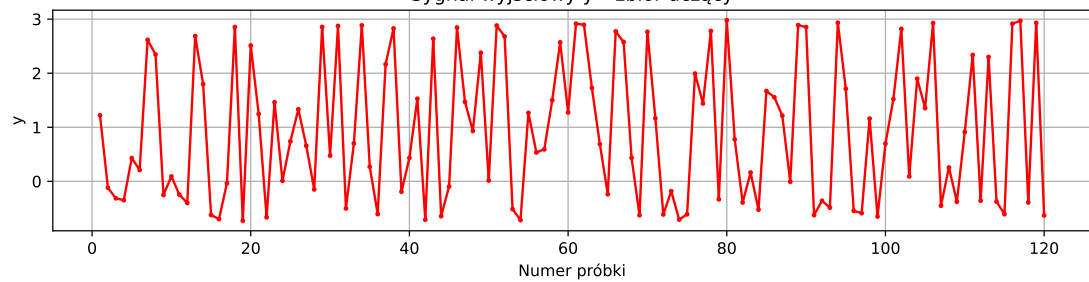


Zbiór uczący - 60 % próbek ( $N_{ucz} = 120$ )

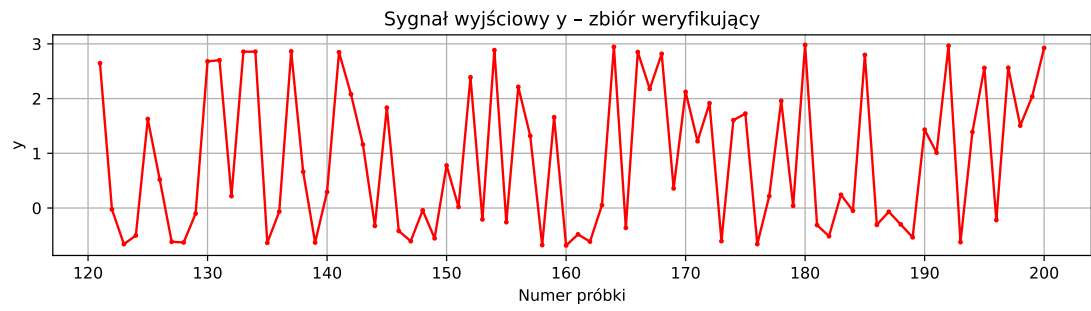
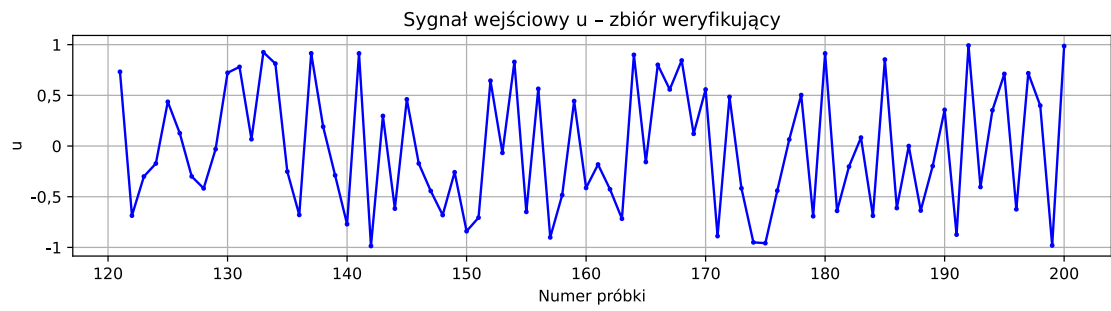
Sygnał wejściowy  $u$  - zbiór uczący



Sygnał wyjściowy  $y$  - zbiór uczący



Zbiór weryfikujący - 40 % próbek ( $N_{\text{wer}} = 80$ )





## 1b. Statyczny model liniowy

Rozważany model ma postać:

$$y(u) = a_0 + a_1 u. \quad (3)$$

Wektor parametrów  $[a_0, a_1]^T$  wyznaczono metodą MNK na zbiorze uczącym, a jakość oceniono osobno dla danych uczących i weryfikujących.

### Listing 2. Statyczny model liniowy i wielomianowy (modele.py)

```
class LinearModel(BaseModel):
    """y(u) = a0 + a1·u"""
    def _build_regressor(self, u: np.ndarray) -> np.ndarray:
        return np.column_stack([np.ones(len(u)), u]) # [1, u]

class PolynomialModel(BaseModel):
    """y(u) = a0 + a1·u + ... + aN·u^N"""
    def __init__(self, degree: int):
        self.degree = degree

    def _build_regressor(self, u: np.ndarray) -> np.ndarray:
        return np.column_stack([u ** i for i in range(self.degree + 1)])

# Wspólna klasa bazowa - uczenie i predykcja przez MNK
class BaseModel:
    def fit(self, u_ucz, y_ucz):
        self._theta = MNKSolver.solve(self._build_regressor(u_ucz),
y_ucz)
        return self

    def predict(self, u):
        return self._build_regressor(u) @ self._theta

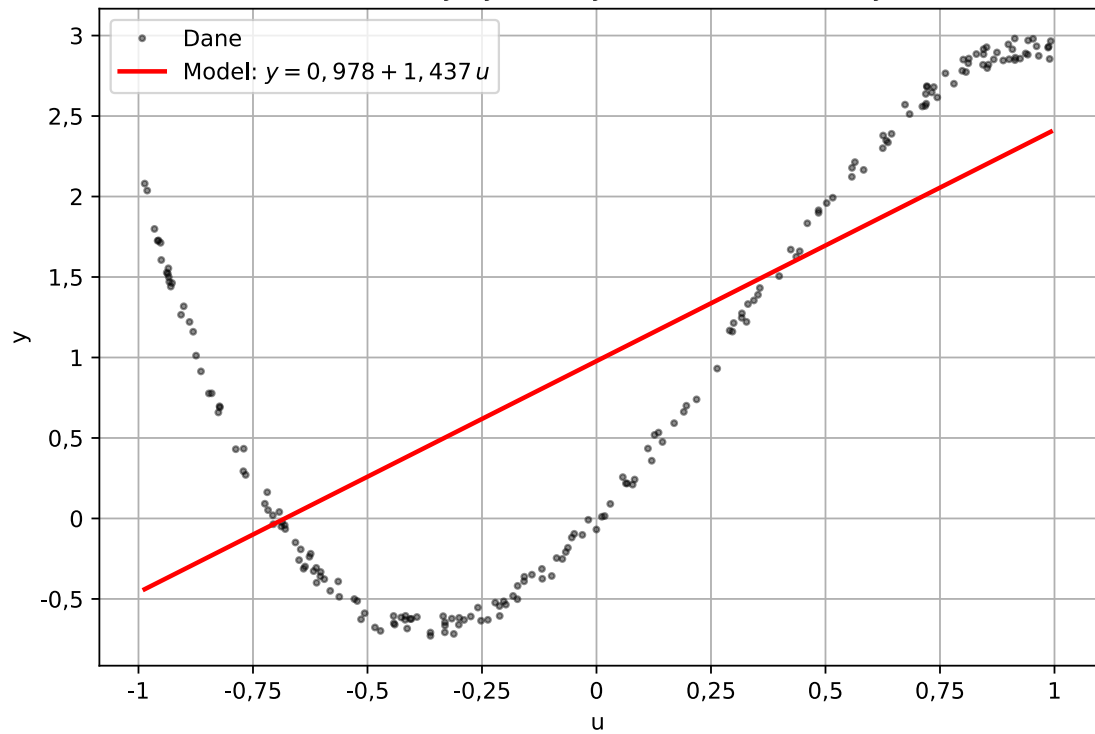
    def evaluate(self, u, y):
        y_pred = self.predict(u)
        mse = float(np.mean((y - y_pred) ** 2))
        return {'y_pred': y_pred, 'mse': mse, 'rmse':
float(np.sqrt(mse))}
```

Model liniowy:  $y(u) = 0,977620 + 1,436710u$

#### Metryki modelu liniowego

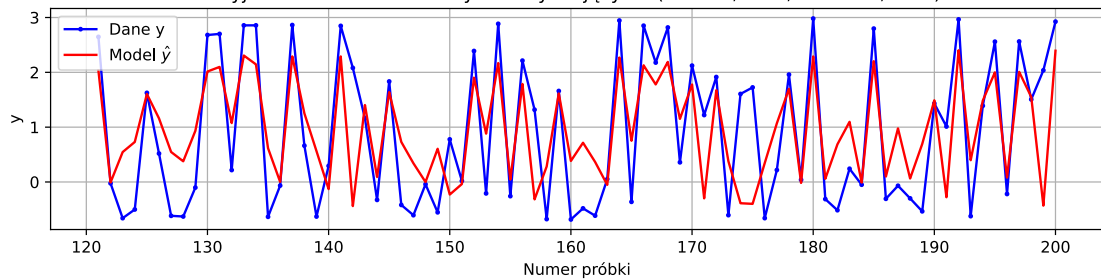
| Model       | MSE ucz. | RMSE ucz. | MSE wer. | RMSE wer. |
|-------------|----------|-----------|----------|-----------|
| liniowy N=1 | 0,897856 | 0,947552  | 0,844686 | 0,919068  |

Charakterystyka statyczna - model liniowy

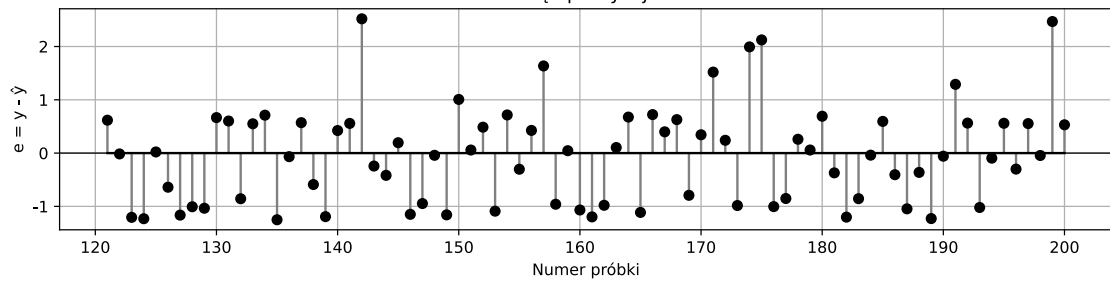


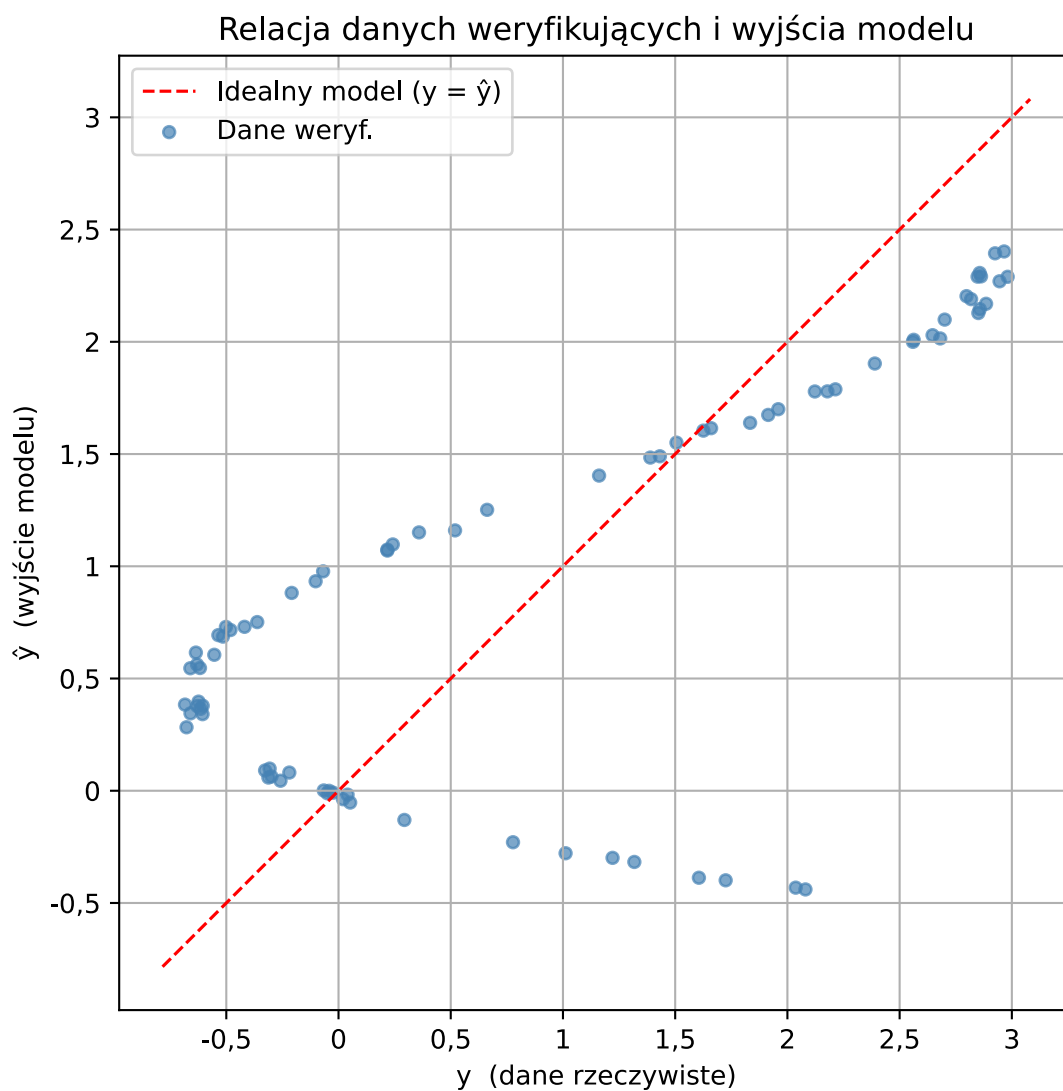
Model liniowy - zbiór weryfikujący

Wyjście modelu na tle danych weryfikujących (MSE=0,8447, RMSE=0,9191)



Błąd predykcji





**Komentarz:** model liniowy stanowi punkt odniesienia. Jeżeli rozrzut punktów na wykresie  $y$  względem  $\hat{y}$  jest systematycznie zakrzywiony, oznacza to, że sama zależność liniowa nie opisuje wystarczająco nieliniowej charakterystyki statycznej.

## 1c. Statyczne modele wielomianowe

Testowany model wielomianowy ma postać:

$$y(u) = a_0 + \sum_{i=1}^N a_i u^i. \quad (4)$$

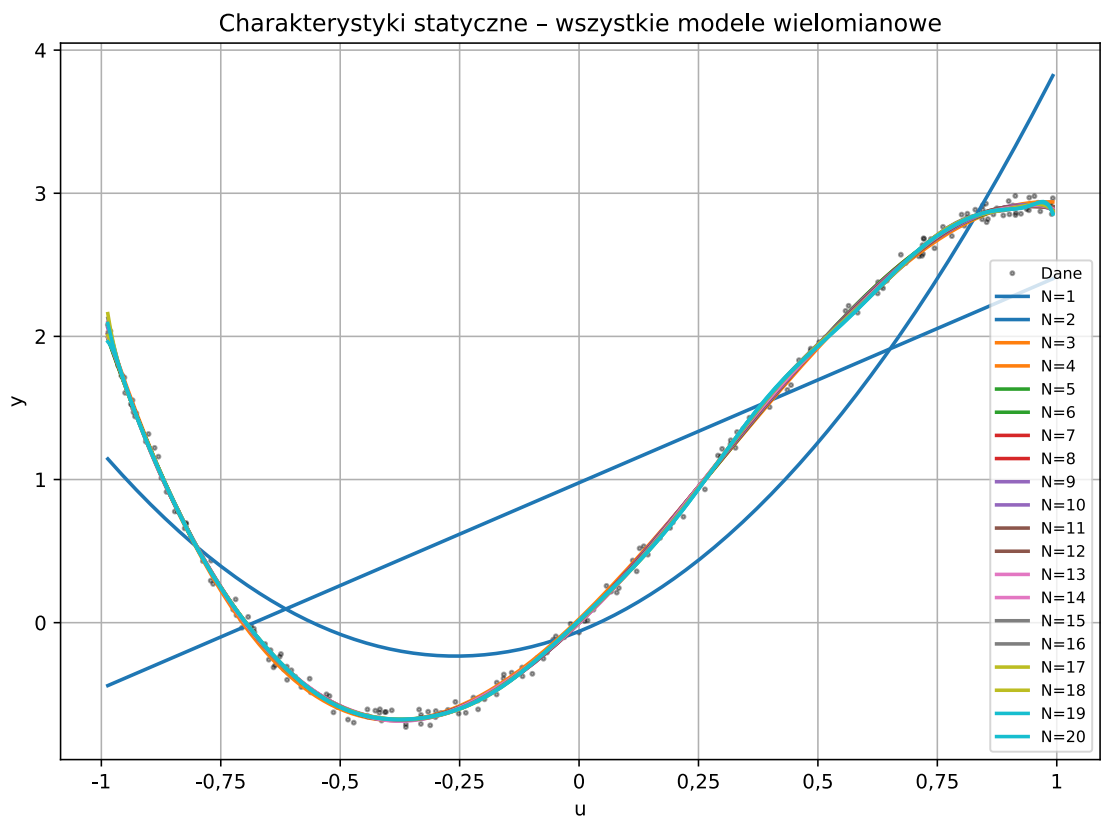
Stopnie wielomianu można edytować w zmiennej `STATIC_POLY_DEGREES` powyżej.

Najlepszy model wybieramy na podstawie najmniejszego błędu na zbiorze weryfikującym, zwracając uwagę na ryzyko przeuczenia przy wysokich stopniach.

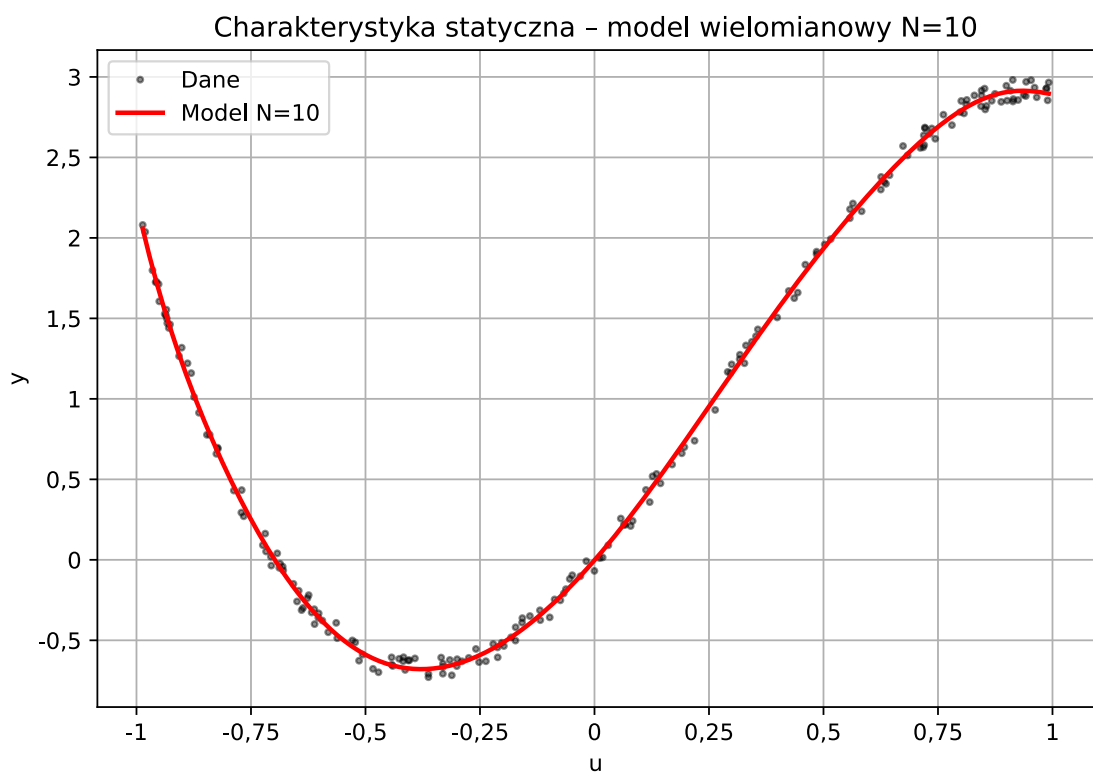
### Metryki statycznych modeli wielomianowych

| N  | L. par. | MSE ucz. | RMSE ucz. | MSE wer. | RMSE wer. |
|----|---------|----------|-----------|----------|-----------|
| 1  | 2       | 0,897856 | 0,947552  | 0,844686 | 0,919068  |
| 2  | 3       | 0,191942 | 0,438112  | 0,207590 | 0,455620  |
| 3  | 4       | 0,002217 | 0,047085  | 0,002472 | 0,049715  |
| 4  | 5       | 0,001903 | 0,043619  | 0,002290 | 0,047859  |
| 5  | 6       | 0,001899 | 0,043574  | 0,002318 | 0,048142  |
| 6  | 7       | 0,001896 | 0,043547  | 0,002273 | 0,047674  |
| 7  | 8       | 0,001869 | 0,043236  | 0,002271 | 0,047651  |
| 8  | 9       | 0,001868 | 0,043218  | 0,002282 | 0,047766  |
| 9  | 10      | 0,001865 | 0,043186  | 0,002270 | 0,047648  |
| 10 | 11      | 0,001848 | 0,042987  | 0,002167 | 0,046549  |
| 11 | 12      | 0,001832 | 0,042806  | 0,002221 | 0,047126  |
| 12 | 13      | 0,001832 | 0,042806  | 0,002229 | 0,047207  |
| 13 | 14      | 0,001792 | 0,042333  | 0,002172 | 0,046600  |
| 14 | 15      | 0,001740 | 0,041711  | 0,002494 | 0,049935  |
| 15 | 16      | 0,001736 | 0,041666  | 0,002385 | 0,048840  |
| 16 | 17      | 0,001718 | 0,041454  | 0,002319 | 0,048156  |
| 17 | 18      | 0,001717 | 0,041440  | 0,002339 | 0,048368  |
| 18 | 19      | 0,001703 | 0,041263  | 0,002502 | 0,050025  |
| 19 | 20      | 0,001700 | 0,041226  | 0,002414 | 0,049137  |
| 20 | 21      | 0,001695 | 0,041169  | 0,002762 | 0,052556  |

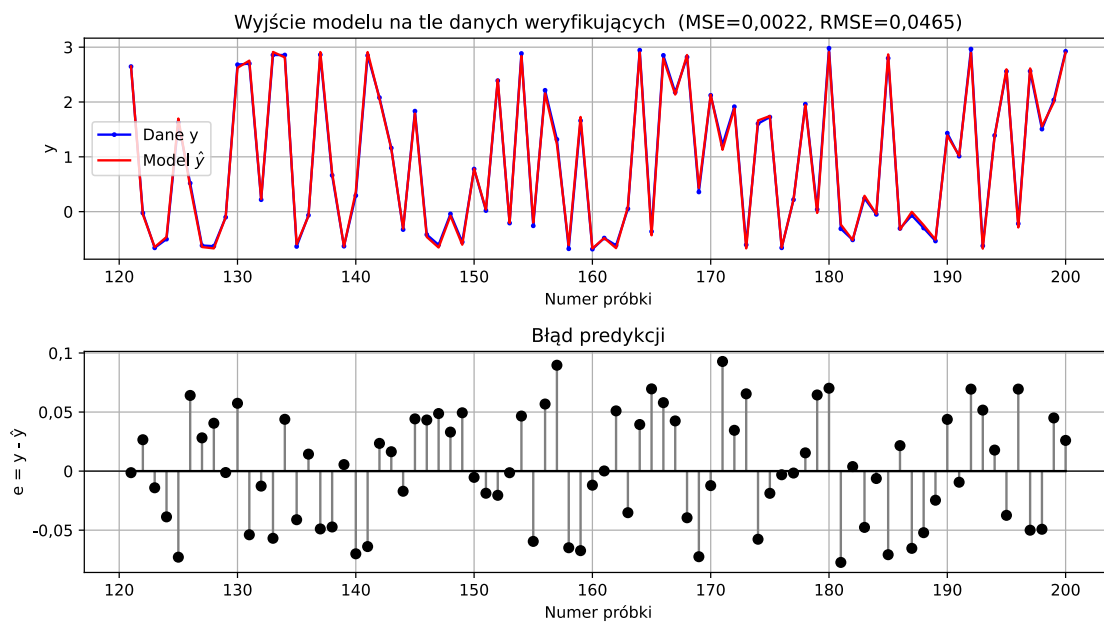
Najniższy błąd weryfikacji uzyskano dla modelu wielomianowego **N=10** (MSE=0.002167, RMSE=0.046549).



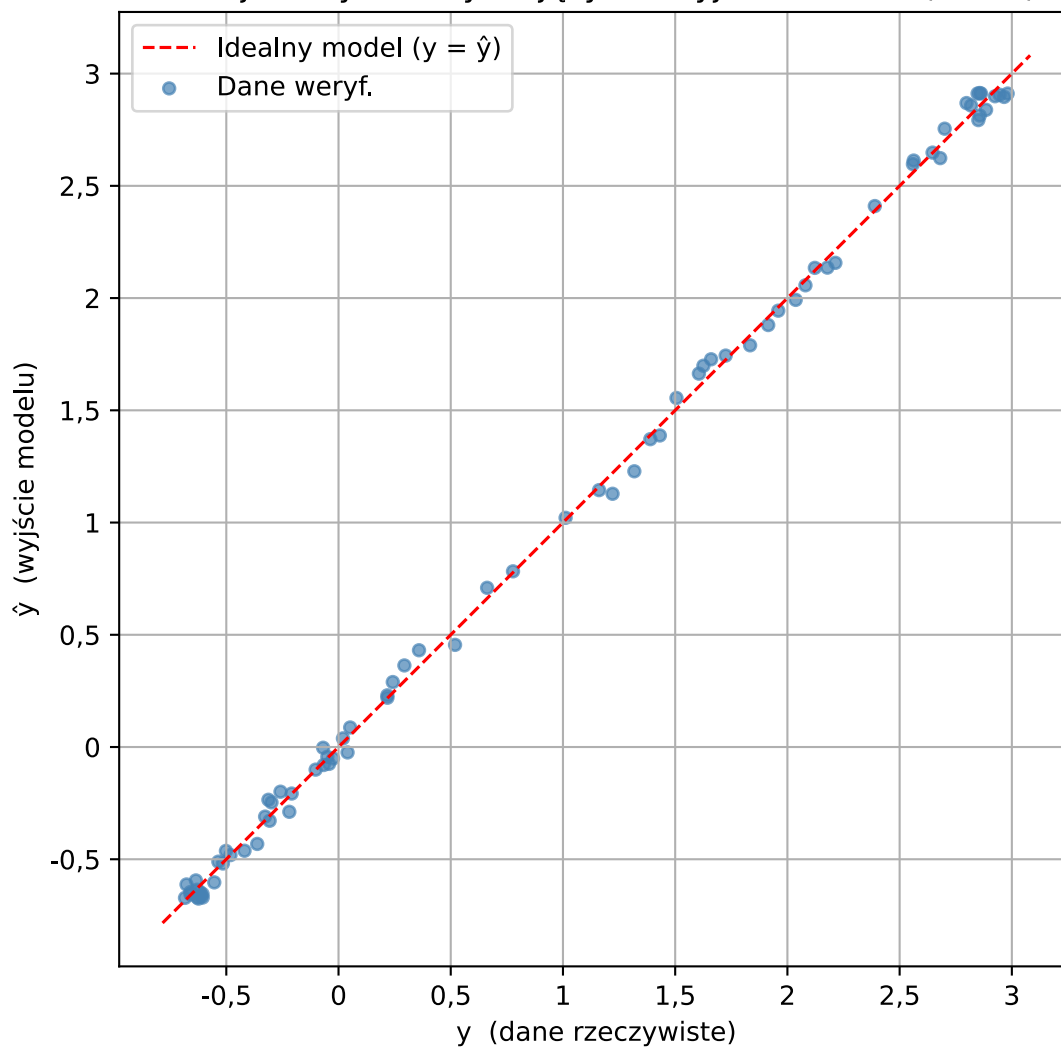
## Wykresy najlepszego modelu statycznego: N=10



### Model wielomianowy N=10 – zbiór weryfikujący



### Relacja danych weryfikujących i wyjścia modelu (N=10)



**Wybór modelu statycznego:** za najlepszy przyjęto model o najmniejszym MSE na zbiorze weryfikującym. Porównanie błędów uczących i weryfikujących pozwala sprawdzić, czy wzrost stopnia wielomianu poprawia uogólnianie, czy tylko dopasowanie do danych uczących.



## 2. Identyfikacja modeli dynamicznych

Dane dynamiczne są dostarczone w dwóch plikach: zbiór uczący i zbiór weryfikujący.

Modele dynamiczne porównano w trybie bez rekurencji oraz z rekurencją. Do wyboru końcowego istotny jest przede wszystkim tryb rekurencyjny, bo odpowiada pracy modelu jako symulatora.

```
In [11]: dynamic_import = load_module("modi_dynamic_import", DYNAMIC_CODE / "import_danyc
dynamic_models = load_module("modi_dynamic_models", DYNAMIC_CODE / "modele_dynam
dynamic_plotting = load_module("modi_dynamic_plotting", DYNAMIC_CODE / "rysowani

ARX_ORDERS = [1, 2, 3]

# Etap 1: szerokie przeszukiwanie, odpowiadające ustawieniu pretrained = False.
pretrained = False
if pretrained == False:
    NARX_PRETRAIN_ORDERS = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50]
    NARX_PRETRAIN_DEGREES = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50]

# Etap 2: przeszukiwanie zawężone, odpowiadające ustawieniu pretrained = True.
pretrained = True
if pretrained == True:
    NARX_ORDERS = [13, 14, 15, 16, 17, 18, 19, 20]
    NARX_DEGREES = [3, 4, 5, 6]

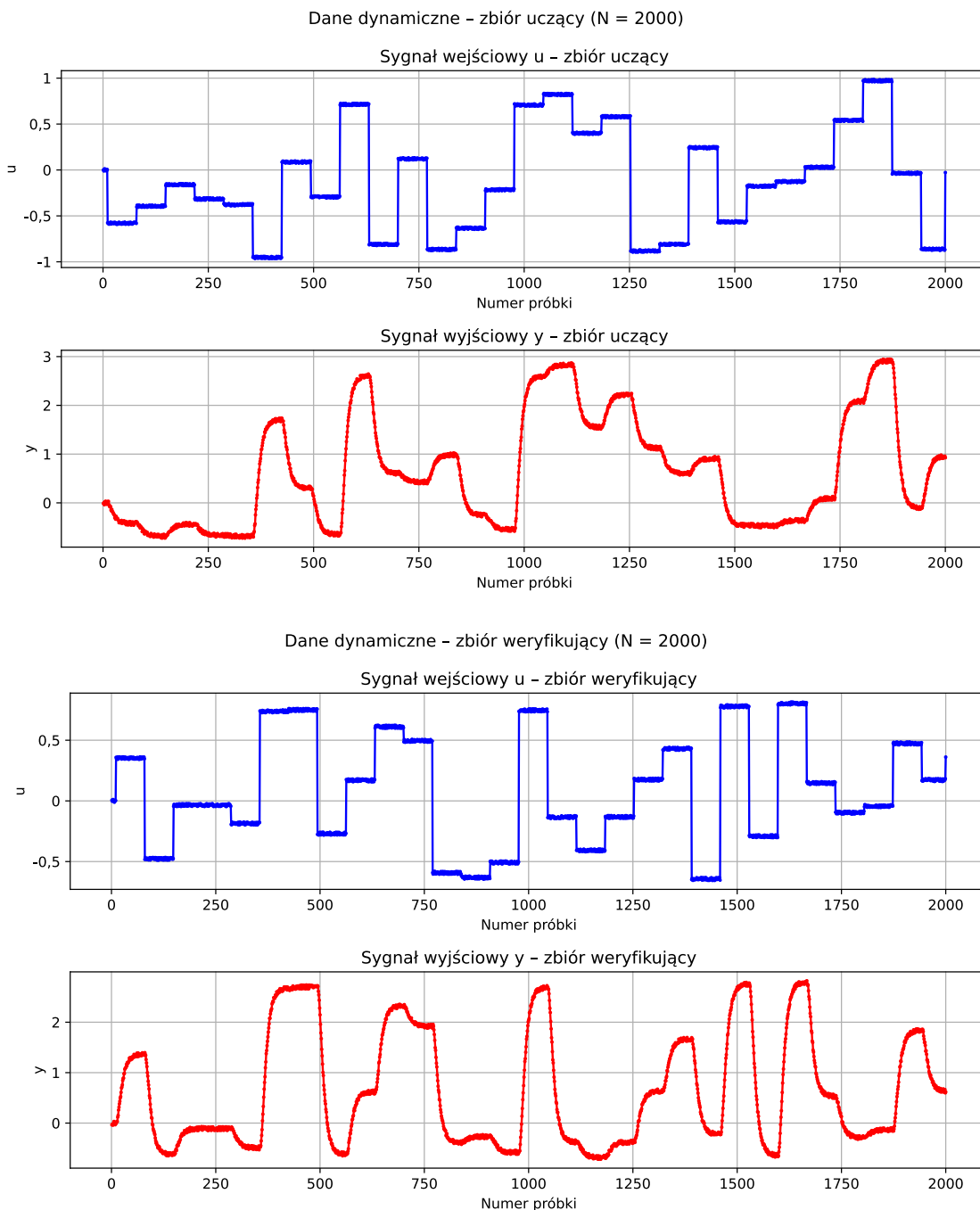
dane_dyn = dynamic_import.ImportDynamicData(base_dir=str(ROOT))
plotter_dyn = dynamic_plotting.PlotDynamic(dane_dyn.out_dir)

if GENERATE_NOTEBOOK_FIGURES:
    plotter_dyn.plot_training_data(dane_dyn.u_ucz, dane_dyn.y_ucz)
    plotter_dyn.plot_verification_data(dane_dyn.u_wer, dane_dyn.y_wer)
    plt.close("all")
```



## 2a. Dane dynamiczne

Na rysunkach 1 i 2 pokazano przebiegi i wyjściowych dla zbioru uczącego oraz weryfikującego.



## 2b. Dynamiczne modele liniowe ARX

Dla rzędów  $n_A = n_B \in \{1, 2, 3\}$  wyznaczono modele:

$$y(k) = \sum_{i=1}^{n_B} b_i u(k-i) + \sum_{i=1}^{n_A} a_i y(k-i). \quad (5)$$

Tryb bez rekurencji używa zmierzonych opóźnionych wartości  $y$ , natomiast tryb rekurencyjny używa wcześniejszych wartości wyjścia modelu  $\hat{y}$ .

### Listing 3. Model ARX – budowa macierzy regresorów i predykcja rekurencyjna

( modele\_dynamiczne.py )

```
class ARXModel:
    def _build_regressor(self, u, y):
        """Phi[k] = [u(k-1),...,u(k-nB), y(k-1),...,y(k-nA)]"""
        n = max(self.nA, self.nB)
        rows = len(u) - n
        Phi = np.zeros((rows, self.nB + self.nA))
        for k in range(rows):
            idx = k + n
            Phi[k, :self.nB] = [u[idx - i - 1] for i in range(self.nB)]
            Phi[k, self.nB:] = [y[idx - i - 1] for i in range(self.nA)]
        return Phi, n

    def predict(self, u, y, recursive=False):
        n = max(self.nA, self.nB)
        b, a = self._theta[:self.nB], self._theta[self.nB:]

        if not recursive:                                # tryb bez
rekurencji
            Phi, n = self._build_regressor(u, y)
            return Phi @ self._theta, n

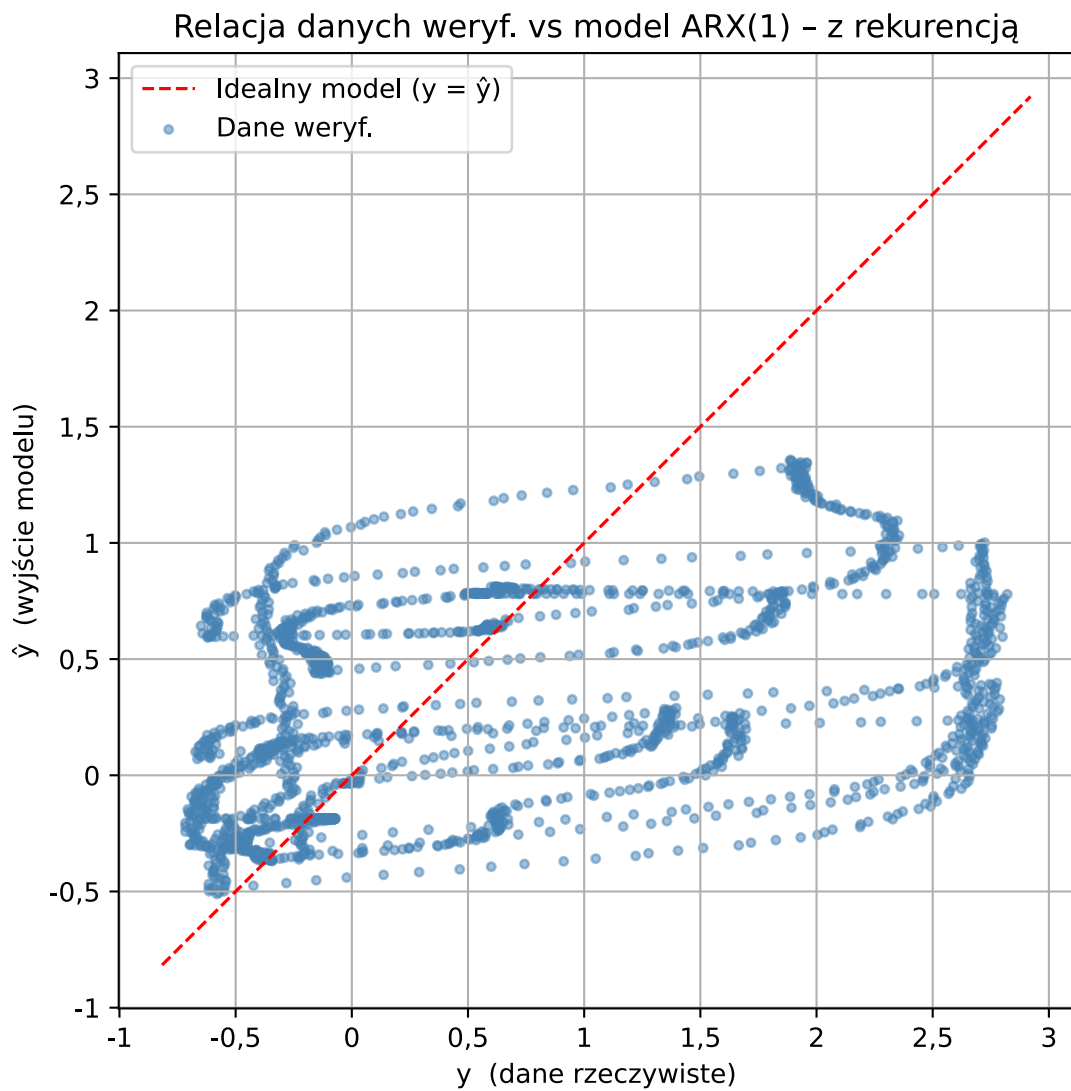
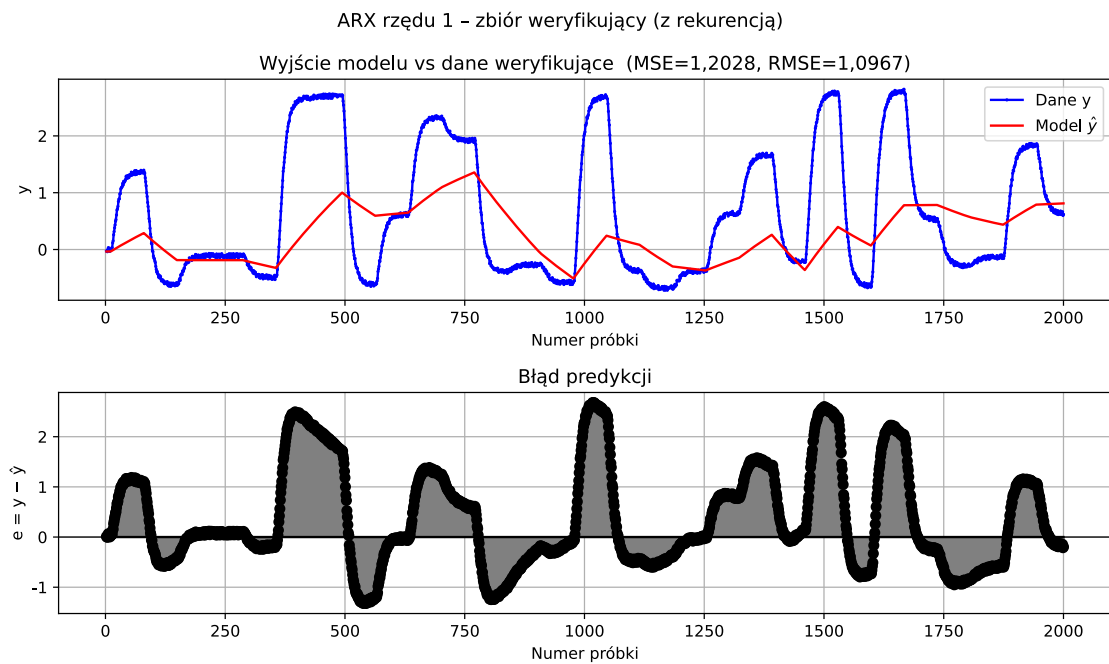
        # tryb rekurencyjny - y-hat(k) korzysta z wcześniejszych y
        y_hat = np.zeros(len(u)); y_hat[:n] = y[:n]
        for k in range(n, len(u)):
            y_hat[k] = (b @ np.array([u[k-i-1] for i in
range(self.nB)]) +
                                a @ np.array([y_hat[k-i-1] for i in
range(self.nA)]))
        return y_hat[n:], n
```

### Metryki modeli ARX

| n | L. par. | MSE ucz. NR | RMSE ucz. NR | MSE wer. NR | RMSE wer. NR | MSE ucz. R | RMSE ucz. R | MSE wer. R | RMSE wer. R |
|---|---------|-------------|--------------|-------------|--------------|------------|-------------|------------|-------------|
| 1 | 2       | 0,001974    | 0,044427     | 0,002631    | 0,051291     | 2,821949   | 1,679866    | 1,202792   | 1,096719    |
| 2 | 4       | 0,001923    | 0,043857     | 0,002384    | 0,048823     | 2,625758   | 1,620419    | 1,213854   | 1,101751    |
| 3 | 6       | 0,001594    | 0,039920     | 0,001798    | 0,042403     | 2,098656   | 1,448674    | 1,219444   | 1,104284    |

Najlepszy model liniowy w trybie rekurencyjnym: **ARX rzędu 1** (MSE=1.202792, RMSE=1.096719).

## Wykresy najlepszego modelu ARX w trybie rekurencyjnym: $n=1$



**Komentarz:** tryb bez rekurencji daje mniejsze błędy, ponieważ korzysta z rzeczywistych opóźnionych wyjść i model w takiej formie działa. Natomiast gdy spróbuje się go użyć praktycznie czyli w trybie rekurencyjnym błędy zaczynają się akumulować. Tryb rekurencyjny ujawnia rzeczywistą przydatność modelu do symulacji procesu i jest ona wątpliwa.

## 2c. Dynamiczne modele nieliniowe NARX

Rozważono wielomianowe modele dynamiczne bez wyrazów mieszanych:

$$y(k) = \sum_{i=1}^{n_B} \sum_{p=1}^d w_{i,p} u(k-i)^p + \sum_{i=1}^{n_A} \sum_{p=1}^d v_{i,p} y(k-i)^p. \quad (6)$$

W tym notebooku zakres przeszukiwania określają listy `NARX_ORDERS` i `NARX_DEGREES`. Liczba parametrów wynosi  $2n_A d$  przy  $n_A = n_B$ .

Przeszukiwanie przeprowadzono **dwuetapowo**:

1. **Etap 1 ( pretrained = False )** – szerokie przeszukiwanie siatki  $n_A \in \{1-10, 20, 30, 40, 50\}$ ,  $d \in \{1-10, 20, 30, 40, 50\}$  w celu ustalenia obiecującego obszaru.
2. **Etap 2 ( pretrained = True )** – zawężone przeszukiwanie wokół minimum znalezione w etapie 1.

**Wybór modelu nieliniowego:** jako kryterium podstawowe przyjęto najmniejsze MSE weryfikacyjne w trybie rekurencyjnym. Jeżeli dwa modele mają bardzo zbliżone błędy, praktycznie korzystniejszy jest model o mniejszej liczbie parametrów, bo jest mniej podatny na przeuczenie i łatwiejszy do interpretacji.

### Etap 1 – wstępne przeszukiwanie ( pretrained = False )

**Listing 4. Model NARX – wektoryzowana budowa macierzy i predykcja rekurencyjna** (modele\_dynamiczne.py)

```
class NARXModel:
    def _build_regressor(self, u, y):
        """Phi[k] = [u(k-1)^1,...,u(k-nB)^d, y(k-1)^1,...,y(k-nA)^d] (bez
        wyrazów mieszanych)"""
        n = max(self.nA, self.nB); rows = len(u) - n
        pows = np.arange(1, self.deg + 1) # [1, 2, ..., deg]

        # macierze opóźnień: shape (rows, nB) i (rows, nA)
        u_lags = np.column_stack([u[n-i-1:n-i-1+rows] for i in
        range(self.nB)])
        y_lags = np.column_stack([y[n-i-1:n-i-1+rows] for i in
        range(self.nA)])

        # Broadcasting (rows, nX, 1) ** (deg,) → (rows, nX*deg) – pełna
        wektoryzacja
        u_pows = (u_lags[:, :, None] ** pows).reshape(rows, self.nB *
        self.deg)
        y_pows = (y_lags[:, :, None] ** pows).reshape(rows, self.nA *
        self.deg)
        return np.hstack([u_pows, y_pows]), n

    def predict(self, u, y, recursive=False):
        n = max(self.nA, self.nB)
        pows = np.arange(1, self.deg + 1)

        if not recursive: # bez rekurencji
            Phi, n = self._build_regressor(u, y)
            return Phi @ self._theta, n

        # tryb rekurencyjny -  $\hat{y}(k)$  używa wcześniejszych  $\hat{y}$  zamiast
        zmierzonych y
        y_hat = np.zeros(len(u)); y_hat[:n] = y[:n]
        for k in range(n, len(u)):
            u_lags_k = np.array([u[k-i-1] for i in range(self.nB)])
            y_lags_k = np.array([y_hat[k-i-1] for i in range(self.nA)])
            phi_k = np.concatenate([
                (u_lags_k[:, None] ** pows).ravel(), # (nB*deg,)
                (y_lags_k[:, None] ** pows).ravel(), # (nA*deg,)
            ])
            val = float(np.dot(phi_k, self._theta))
            y_hat[k] = val if np.isfinite(val) else 0.0
        return y_hat[n:], n
```

In [15]: RUN\_NARX\_PRETRAIN = False # True = przelicz na nowo (wolne, ~196 modeli); Fals

Szerokie przeszukiwanie przeprowadzono jednorazowo na siatce `NARX_PRETRAIN_ORDERS`  
= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50] × `NARX_PRETRAIN_DEGREES`  
= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50] .

**Najlepszy wynik (etap 1):** NARX( $n_A=n_B=20$ ,  $d=4$ ) → MSE wer. R = 0.000573.

Na tej podstawie zawężono zakres do `NARX_ORDERS` = [13, 14, 15, 16, 17, 18, 19, 20] , `NARX_DEGREES` = [3, 4, 5, 6] .

## Etap 2 – zawężone przeszukiwanie ( pretrained = True )

Na podstawie wyników etapu 1 zawężono siatkę do  $n_A \in \{13-20\}$ ,  $d \in \{3, 4, 5, 6\}$  i powtórzono przeszukiwanie.

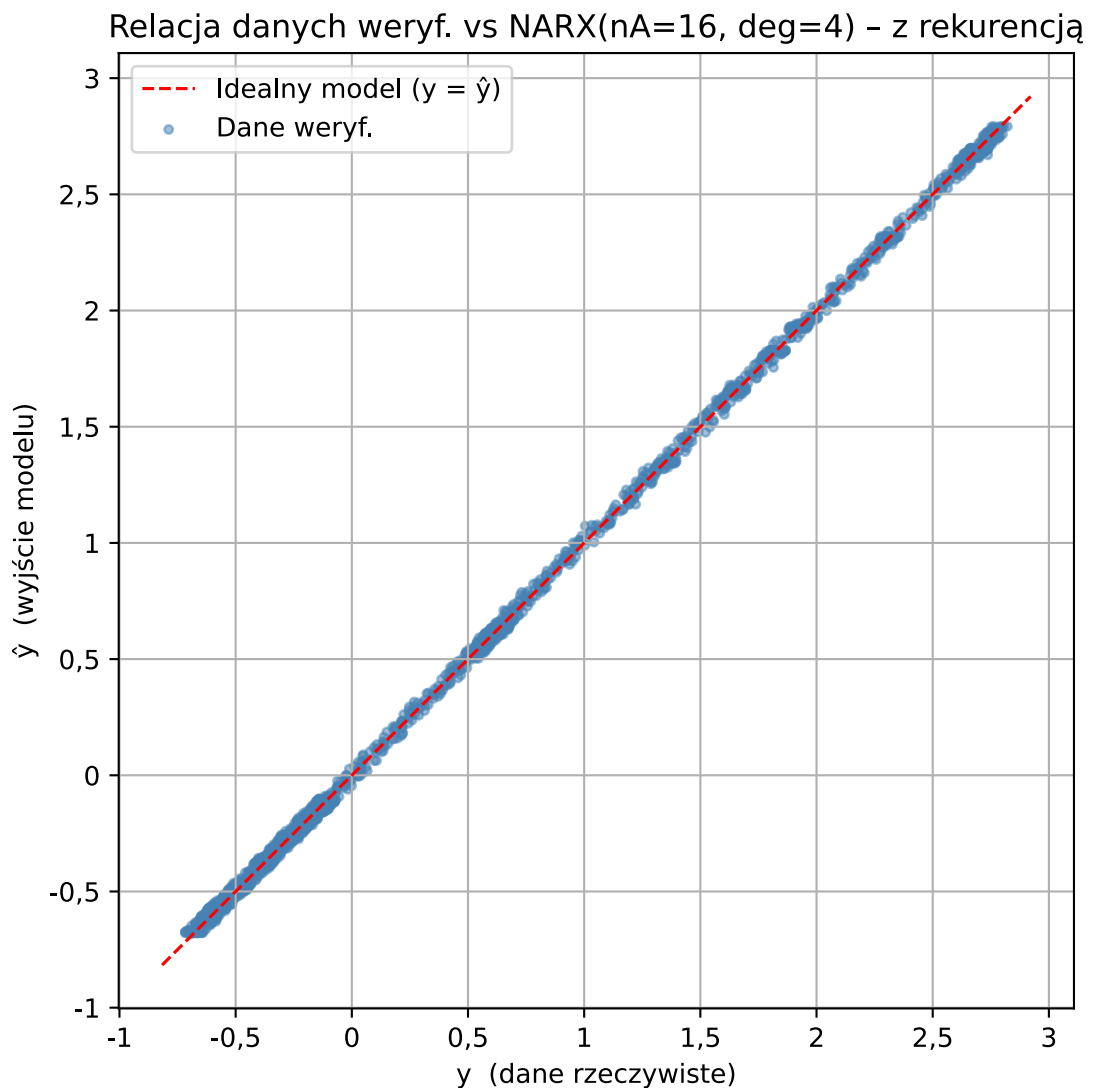
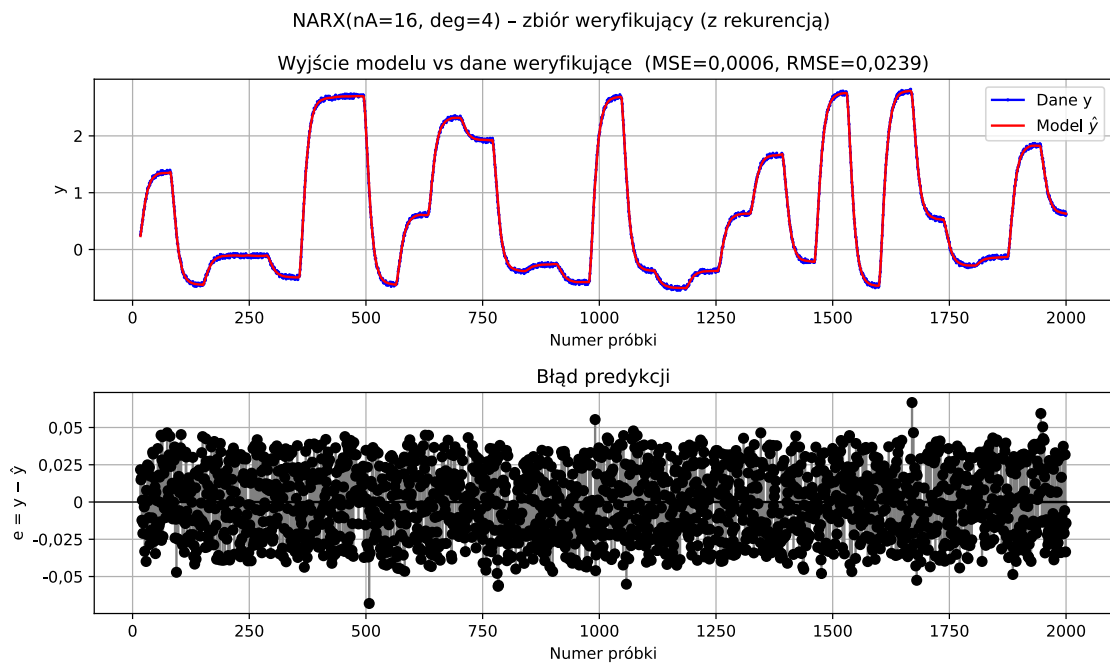
### Metryki modeli NARX

| $n_A=n_B$ | $d$ | L. par. | MSE ucz. NR | RMSE ucz. NR | MSE wer. NR | RMSE wer. NR | MSE ucz. R | RMSE ucz. R | MSE wer. R | RMSE wer. R |
|-----------|-----|---------|-------------|--------------|-------------|--------------|------------|-------------|------------|-------------|
| 13        | 3   | 78      | 0,000582    | 0,024120     | 0,000620    | 0,024905     | 0,000735   | 0,027106    | 0,000725   | 0,026924    |
| 13        | 4   | 104     | 0,000559    | 0,023643     | 0,000615    | 0,024802     | 0,000556   | 0,023570    | 0,000589   | 0,024268    |
| 13        | 5   | 130     | 0,000552    | 0,023496     | 0,000623    | 0,024957     | 0,000555   | 0,023555    | 0,000598   | 0,024449    |
| 13        | 6   | 156     | 0,000544    | 0,023324     | 0,000631    | 0,025118     | 0,000555   | 0,023550    | 0,000614   | 0,024783    |
| 14        | 3   | 84      | 0,000576    | 0,023996     | 0,000615    | 0,024804     | 0,000733   | 0,027069    | 0,000713   | 0,026693    |
| 14        | 4   | 112     | 0,000548    | 0,023412     | 0,000611    | 0,024728     | 0,000545   | 0,023353    | 0,000581   | 0,024107    |
| 14        | 5   | 140     | 0,000541    | 0,023266     | 0,000622    | 0,024933     | 0,000544   | 0,023327    | 0,000594   | 0,024362    |
| 14        | 6   | 168     | 0,000532    | 0,023060     | 0,000632    | 0,025137     | 0,000540   | 0,023246    | 0,000610   | 0,024699    |
| 15        | 3   | 90      | 0,000569    | 0,023862     | 0,000610    | 0,024689     | 0,000732   | 0,027055    | 0,000708   | 0,026611    |
| 15        | 4   | 120     | 0,000536    | 0,023142     | 0,000605    | 0,024598     | 0,000535   | 0,023124    | 0,000580   | 0,024078    |
| 15        | 5   | 150     | 0,000529    | 0,022994     | 0,000619    | 0,024883     | 0,000532   | 0,023057    | 0,000592   | 0,024327    |
| 15        | 6   | 180     | 0,000518    | 0,022769     | 0,000631    | 0,025113     | 0,000528   | 0,022970    | 0,000605   | 0,024603    |
| 16        | 3   | 96      | 0,000566    | 0,023793     | 0,000610    | 0,024695     | 0,000735   | 0,027106    | 0,000699   | 0,026431    |
| 16        | 4   | 128     | 0,000529    | 0,023001     | 0,000603    | 0,024555     | 0,000529   | 0,023001    | 0,000570   | 0,023884    |
| 16        | 5   | 160     | 0,000521    | 0,022834     | 0,000615    | 0,024807     | 0,000524   | 0,022896    | 0,000578   | 0,024045    |
| 16        | 6   | 192     | 0,000511    | 0,022598     | 0,000629    | 0,025082     | 0,000520   | 0,022796    | 0,000592   | 0,024335    |
| 17        | 3   | 102     | 0,000563    | 0,023729     | 0,000609    | 0,024677     | 0,000731   | 0,027042    | 0,000697   | 0,026405    |
| 17        | 4   | 136     | 0,000522    | 0,022845     | 0,000600    | 0,024489     | 0,000526   | 0,022935    | 0,000572   | 0,023907    |
| 17        | 5   | 170     | 0,000513    | 0,022656     | 0,000612    | 0,024738     | 0,000520   | 0,022802    | 0,000579   | 0,024071    |
| 17        | 6   | 204     | 0,000502    | 0,022401     | 0,000628    | 0,025051     | 0,000515   | 0,022683    | 0,000594   | 0,024370    |
| 18        | 3   | 108     | 0,000560    | 0,023667     | 0,000612    | 0,024735     | 0,000728   | 0,026986    | 0,000695   | 0,026359    |
| 18        | 4   | 144     | 0,000517    | 0,022747     | 0,000599    | 0,024478     | 0,000524   | 0,022890    | 0,000571   | 0,023903    |
| 18        | 5   | 180     | 0,000509    | 0,022550     | 0,000613    | 0,024749     | 0,000518   | 0,022755    | 0,000580   | 0,024078    |
| 18        | 6   | 216     | 0,000497    | 0,022290     | 0,000628    | 0,025055     | 0,000512   | 0,022626    | 0,000591   | 0,024317    |
| 19        | 3   | 114     | 0,000559    | 0,023649     | 0,000610    | 0,024707     | 0,000728   | 0,026982    | 0,000693   | 0,026325    |
| 19        | 4   | 152     | 0,000513    | 0,022655     | 0,000598    | 0,024449     | 0,000522   | 0,022846    | 0,000572   | 0,023916    |
| 19        | 5   | 190     | 0,000504    | 0,022454     | 0,000611    | 0,024714     | 0,000516   | 0,022705    | 0,000580   | 0,024081    |
| 19        | 6   | 228     | 0,000491    | 0,022164     | 0,000626    | 0,025018     | 0,000509   | 0,022555    | 0,000591   | 0,024304    |
| 20        | 3   | 120     | 0,000557    | 0,023604     | 0,000615    | 0,024794     | 0,000731   | 0,027035    | 0,000695   | 0,026371    |
| 20        | 4   | 160     | 0,000509    | 0,022563     | 0,000598    | 0,024457     | 0,000521   | 0,022815    | 0,000573   | 0,023931    |
| 20        | 5   | 200     | 0,000499    | 0,022345     | 0,000615    | 0,024794     | 0,000514   | 0,022663    | 0,000583   | 0,024152    |
| 20        | 6   | 240     | 0,000486    | 0,022044     | 0,000630    | 0,025090     | 0,000507   | 0,022510    | 0,000595   | 0,024396    |

Najlepszy model nieliniowy w trybie rekurencyjnym: **NARX( $n_A=n_B=16$ ,  $d=4$ )** z 128 parametrami (MSE=0,000570, RMSE=0,023884).



## Wykresy najlepszego modelu NARX w trybie rekurencyjnym: $nA=nB=16$ , $d=4$

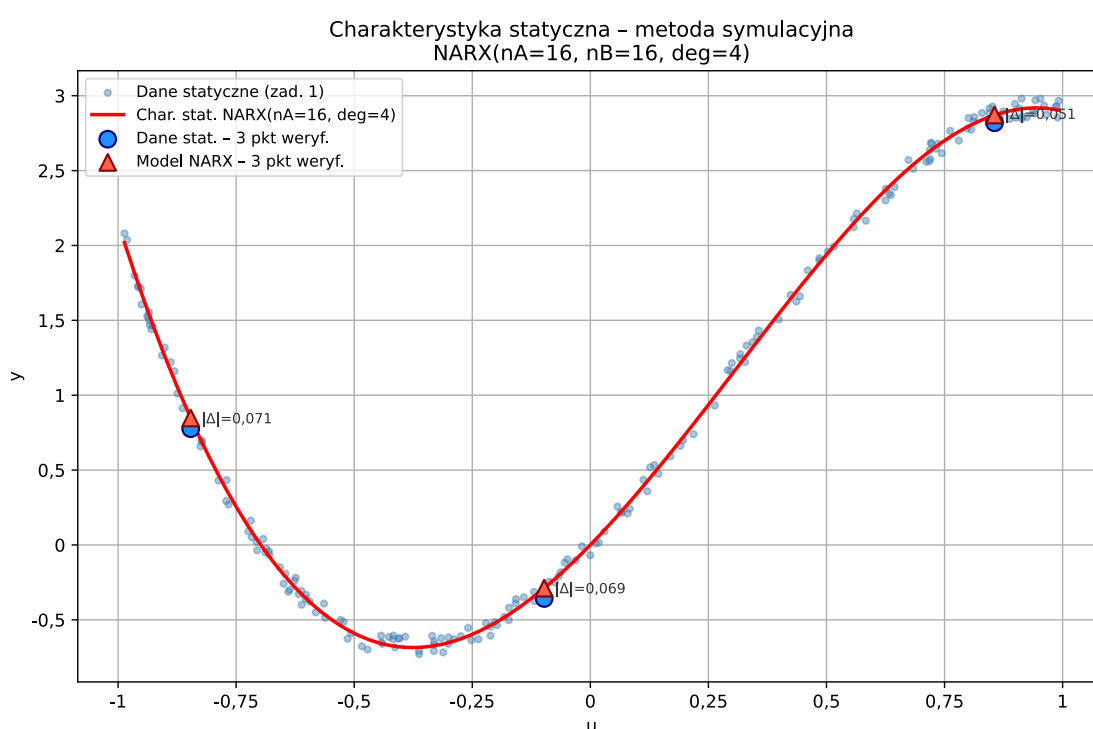


## 2d. Charakterystyka statyczna z modelu NARX

Charakterystykę statyczną najlepszego modelu NARX wyznaczono metodą symulacyjną. Dla stałego wejścia  $u^*$  uruchomiono model rekurencyjnie przez wiele próbek, a ostatnią wartość  $\hat{y}$  potraktowano jako stan ustalony.

### Porównanie charakterystyki statycznej w 3 punktach

| $u^*$     | y statyczne | y NARX    | błąd     |
|-----------|-------------|-----------|----------|
| -0,846223 | 0,776698    | 0,847879  | 0,071180 |
| -0,097748 | -0,357367   | -0,288268 | 0,069098 |
| 0,855879  | 2,819566    | 2,871001  | 0,051434 |



## Wnioski końcowe

1. Model liniowy jest użytecznym punktem odniesienia, ale w części statycznej dokładniejsze wyniki daje model wielomianowy dobrany na podstawie błędu weryfikacyjnego.
2. W modelach dynamicznych ocena rekurencyjna jest surowsza od predykcji bez rekurencji, bo błędy modelu wpływają na kolejne próbki.
3. Najlepszy model NARX dobrany według MSE weryfikacyjnego w trybie rekurencyjnym najlepiej nadaje się do dalszej symulacji i do wyznaczenia charakterystyki statycznej metodą symulacyjną.
4. Zgodność charakterystyki statycznej NARX z danymi statycznymi należy oceniać nie tylko po trzech punktach kontrolnych, ale też po kształcie całej krzywej względem chmury danych.

### 3. Identyfikacja modelem neuronowym (Keras)

Statyczny model neuronowy z **jedną warstwą ukrytą**:

$$\hat{y}(u) = \sum_{j=1}^H v_j \cdot \tanh\left(\sum_{i=1}^1 w_{ji} u + b_j^{(1)}\right) + b^{(2)}, \quad (7)$$

gdzie  $H$  – liczba neuronów ukrytych. Wagi uczone metodą minimalizacji MSE algorytmem wstecznej propagacji błędów.

Rozważono trzy przypadki:

| Przypadek   | Neurony ukryte | Liczba parametrów |
|-------------|----------------|-------------------|
| Zbyt mała   | 1              | 4                 |
| Umiarkowana | 10             | 31                |
| Zbyt duża   | 500            | 1 501             |

Dla każdego przypadku uczenie powtórzono przy zmianach:

- **algorytmu optymalizacji**: Adam, SGD, RMSprop (przy  $lr = 0,01$  i stałym punkcie startowym),
- **współczynnika uczenia**: 0,001 / 0,01 / 0,1 (przy Adam i stałym punkcie startowym),
- **punktu startowego wag**: seed = 42 / 123 / 999 (przy Adam i  $lr = 0,01$ ).

Backend Keras: PyTorch (TensorFlow nie wspiera Pythona 3.14).

**Listing 5. Statyczny model neuronowy Keras** (modele\_neuronowe.py)

```
class KerasNNModel:
    """Input(1) → Dense(n_hidden, tanh) → Dense(1, linear)"""

    def _build(self):
        keras.utils.set_random_seed(self.seed)
        model = keras.Sequential([
            layers.Input(shape=(1,)),
            layers.Dense(self.n_hidden, activation='tanh'),
            layers.Dense(1),          # wyjście liniowe
        ])
        model.compile(
            optimizer=_OPTIMIZERS[self.optimizer_name]
            (learning_rate=self.lr),
            loss='mse',
        )
        return model

    def fit(self, u_ucz, y_ucz, epochs=3000, batch_size=32, verbose=0):
        self._model = self._build()
        self._history = self._model.fit(
            u_ucz.reshape(-1, 1), y_ucz,
            epochs=epochs,
            batch_size=batch_size,
            verbose=verbose,
        )
        return self

    def predict(self, u):
        return self._model.predict(u.reshape(-1, 1), verbose=0).ravel()

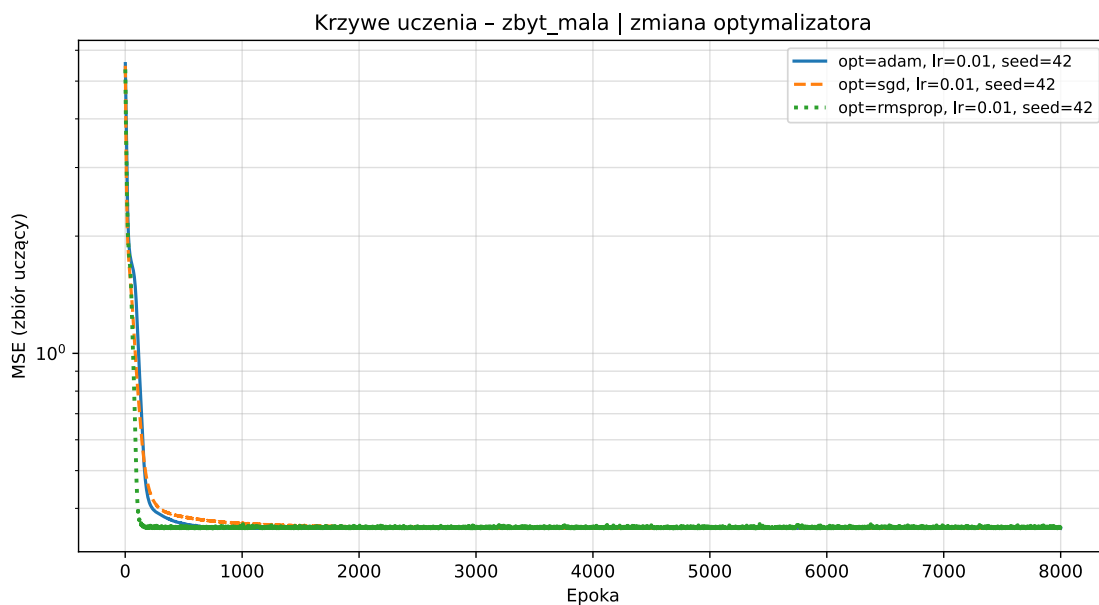
    def evaluate(self, u, y):
        y_pred = self.predict(u)
        mse = float(np.mean((y - y_pred) ** 2))
        return {'y_pred': y_pred, 'mse': mse, 'rmse':
float(np.sqrt(mse))}
```

In [21]: RUN\_NEURAL = False # True = przelicz od nowa (wolne, ~27 modeli); False = wczyt

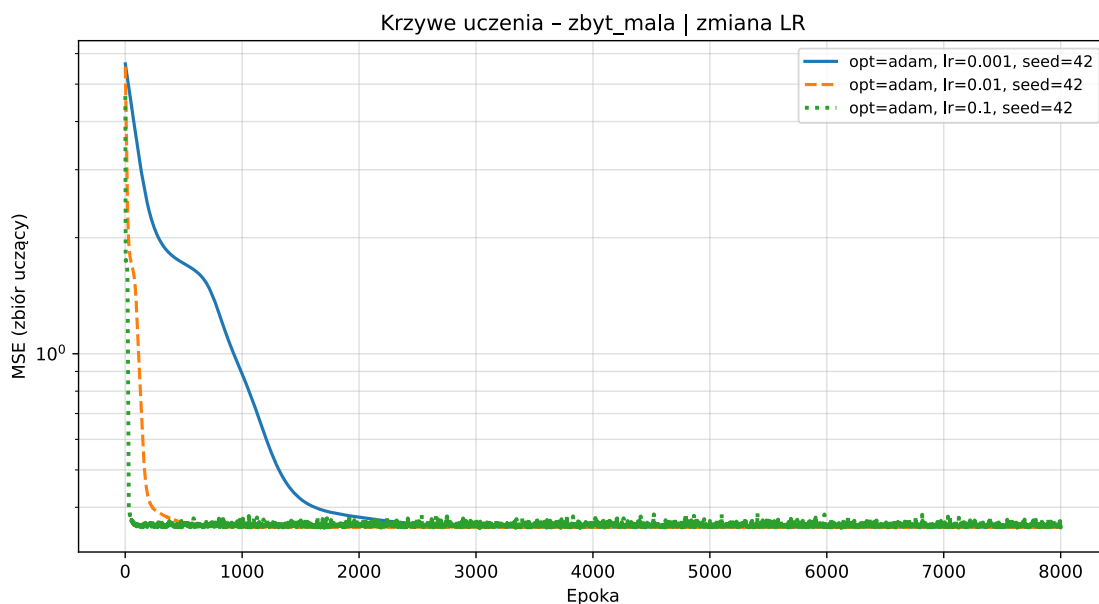
### 3a. Sieć zbyt mała (1 neuron ukryty)

Sieć z jednym neuronem ukrytym ma tylko **4 parametry**. Funkcja  $\tanh$  daje krzywą sigmoidalną – model nie jest w stanie odwzorować złożonej charakterystyki statystycznej.

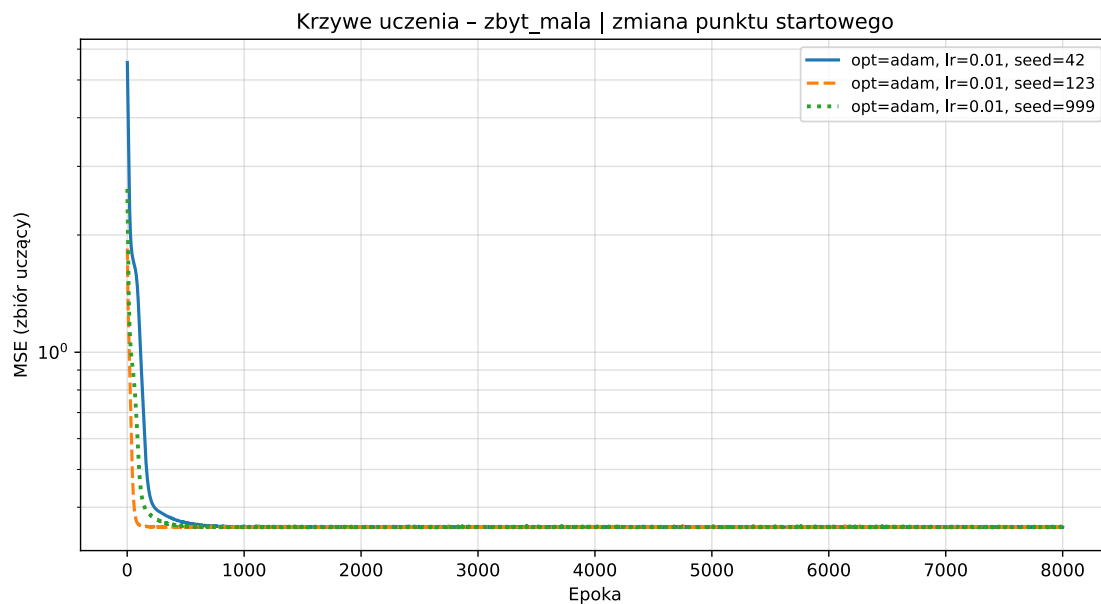
#### Zmiana optymalizatora (lr=0.01, seed=42)



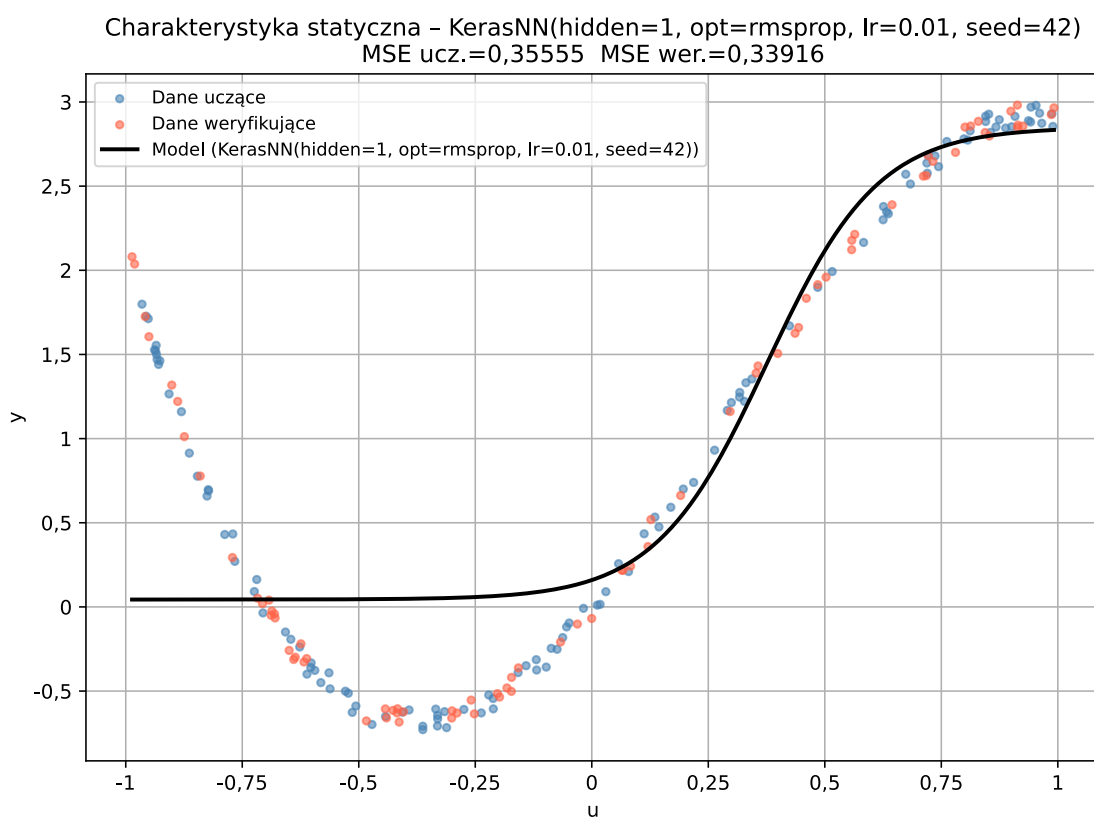
#### Zmiana współczynnika uczenia (Adam, seed=42)



#### Zmiana punktu startowego (Adam, lr=0.01)



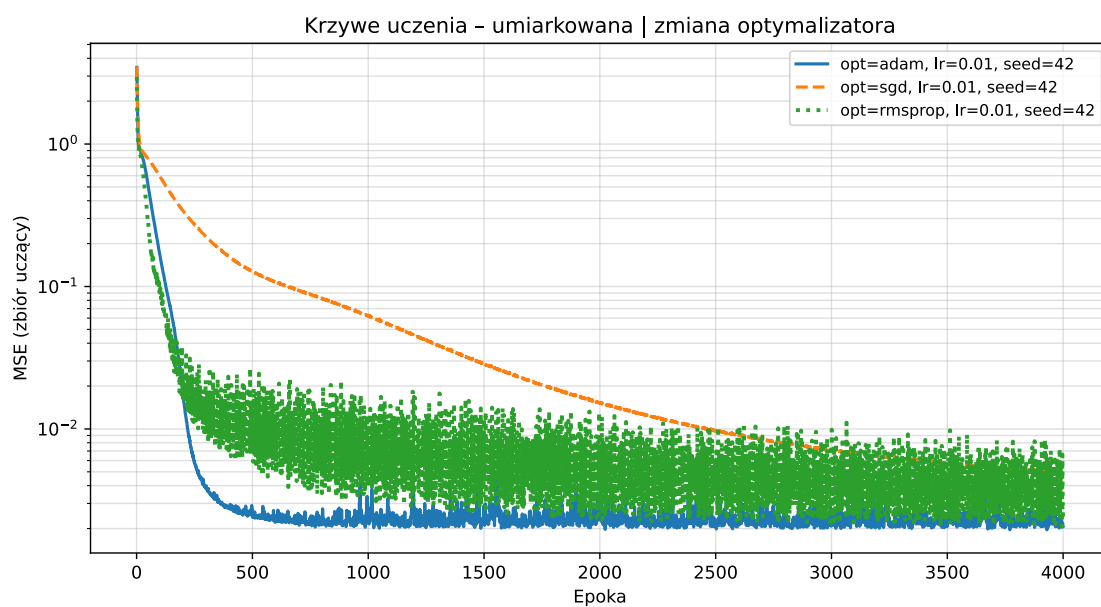
### Charakterystyka statyczna – najlepszy model



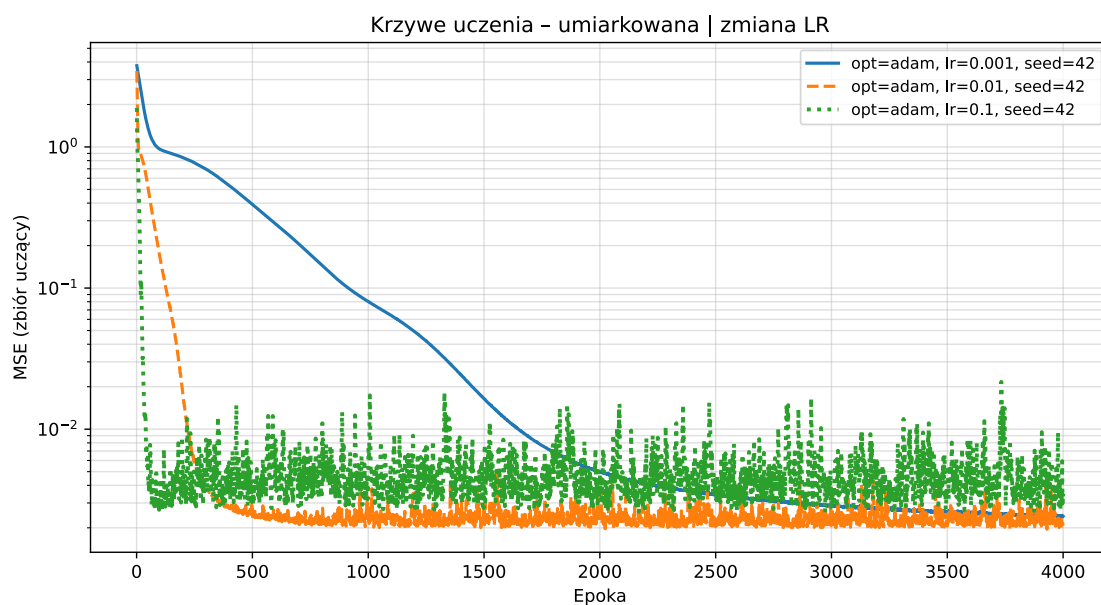
## 3b. Sieć umiarkowana (10 neuronów ukrytych)

Sieć z 10 neuronami ukrytymi ma **31 parametrów**. Powinna być wystarczająco elastyczna do aproksymacji statycznej charakterystyki, przy jednoczesnym braku wyraźnego przeuczenia.

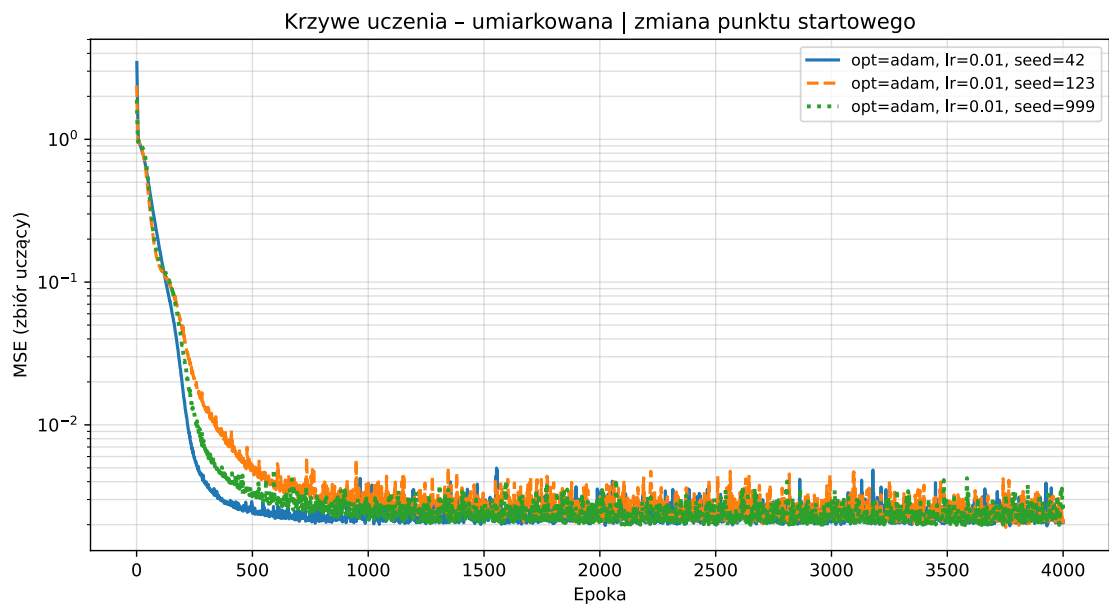
### Zmiana optymalizatora (lr=0.01, seed=42)



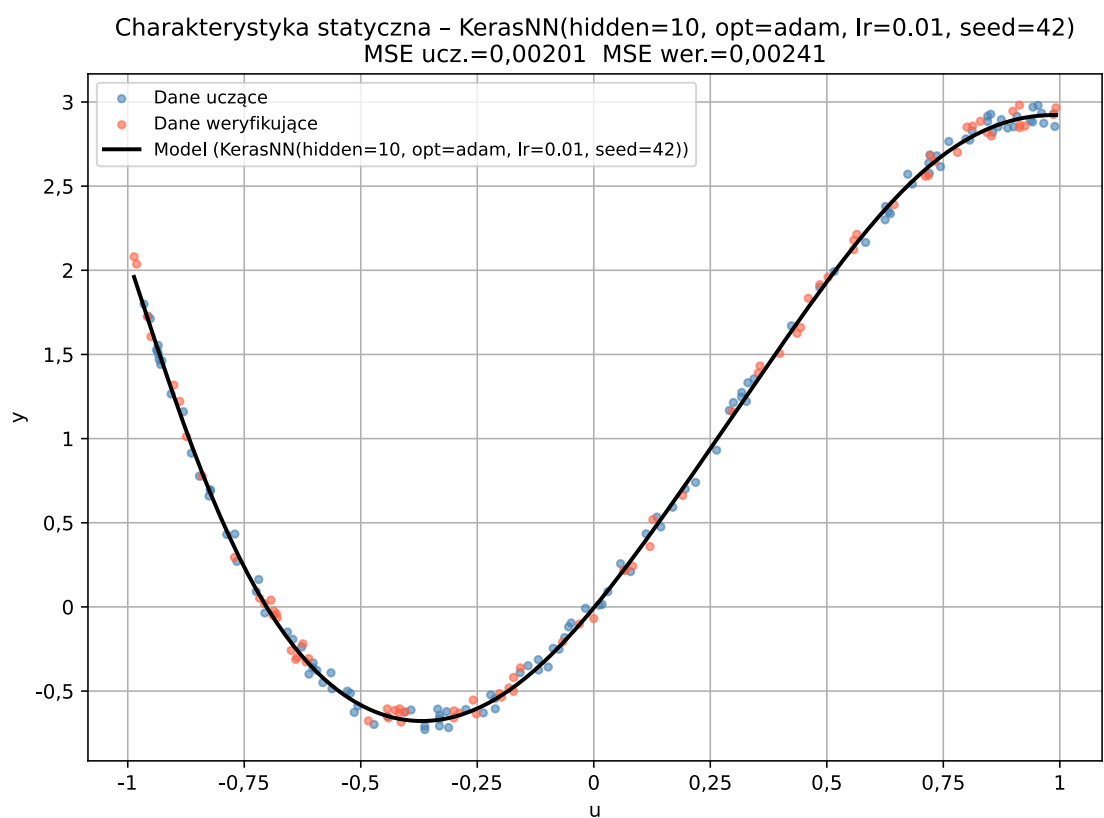
### Zmiana współczynnika uczenia (Adam, seed=42)



### Zmiana punktu startowego (Adam, lr=0.01)



### Charakterystyka statyczna – najlepszy model

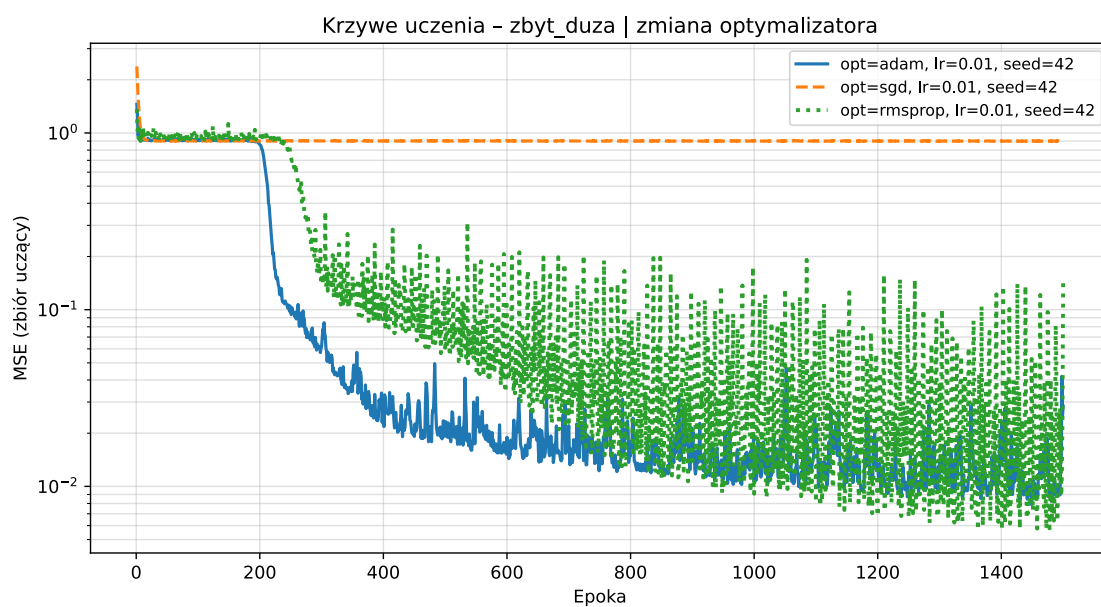




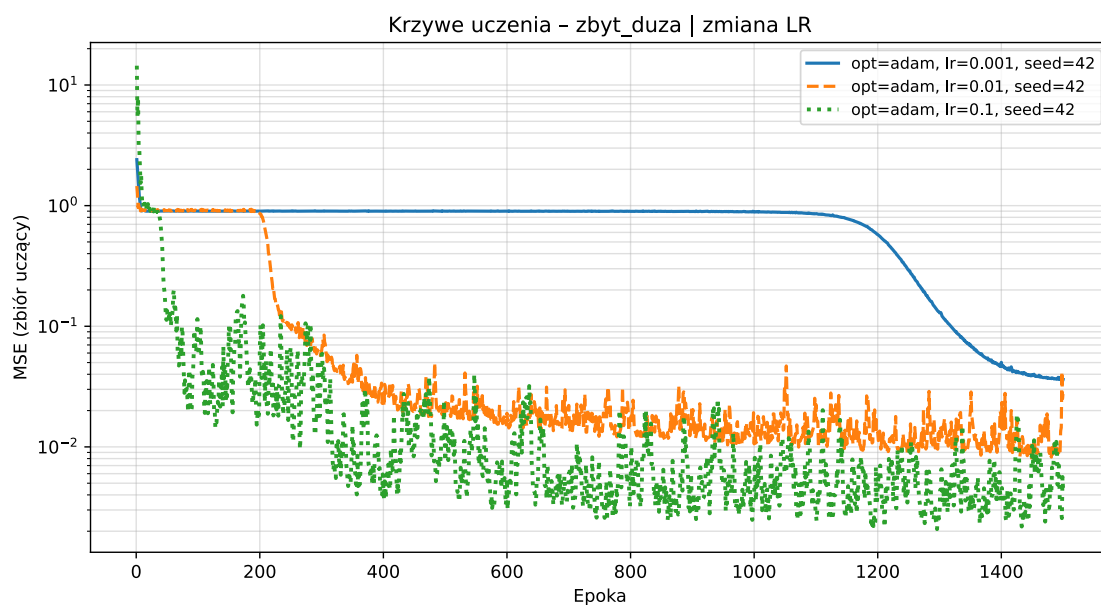
### 3c. Sieć zbyt duża (500 neuronów ukrytych)

Sieć z 500 neuronami ukrytymi ma **1 501 parametrów** – ponad 12-krotnie więcej niż danych uczących. Oczekuje się przeuczenia: MSE uczący będzie bardzo niski, ale MSE weryfikujący może wzrosnąć.

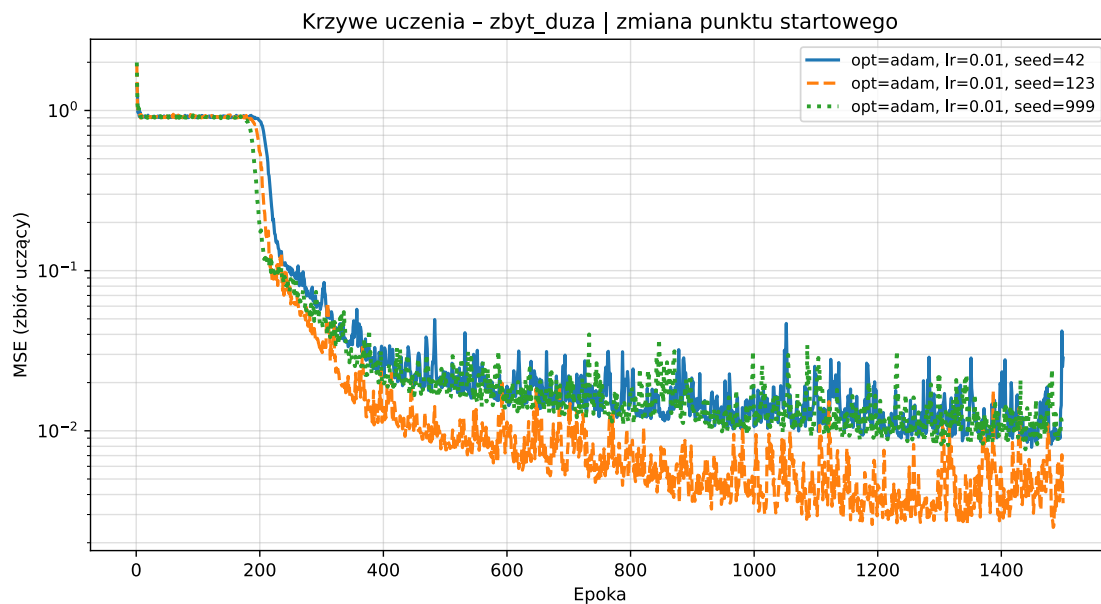
#### Zmiana optymalizatora (lr=0.01, seed=42)



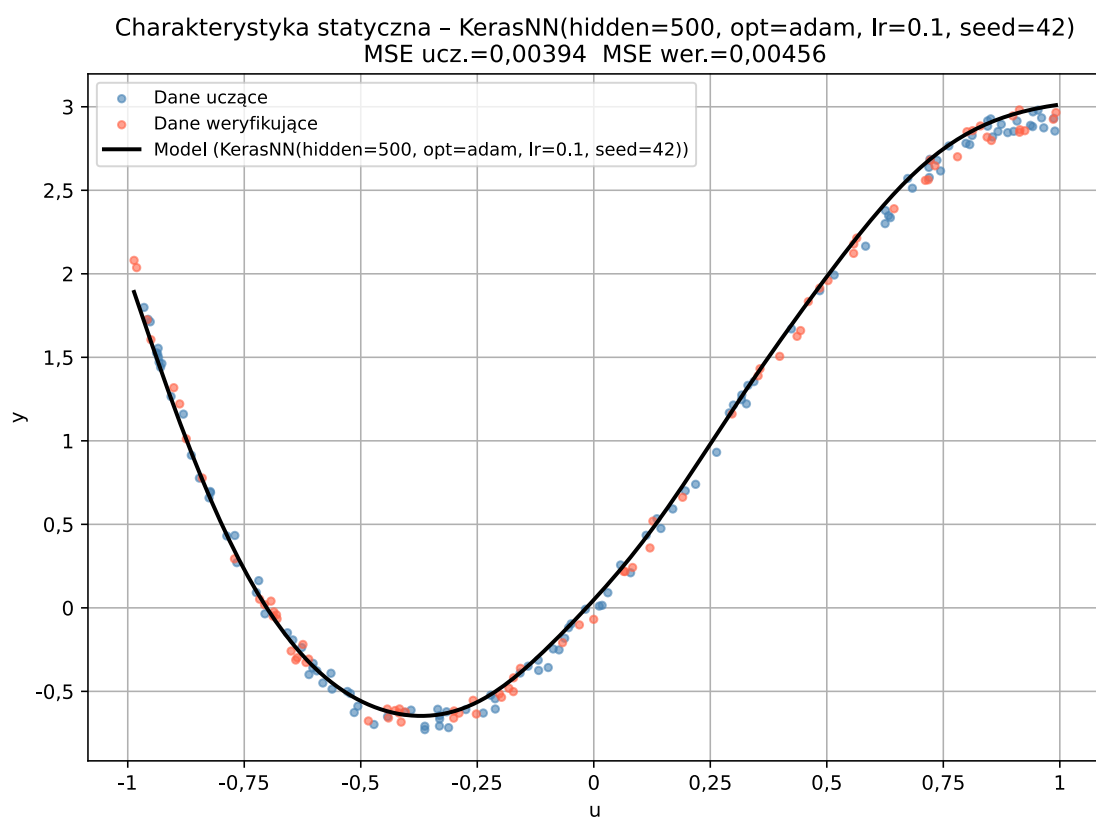
#### Zmiana współczynnika uczenia (Adam, seed=42)



#### Zmiana punktu startowego (Adam, lr=0.01)



### Charakterystyka statyczna – najlepszy model



### 3d. Zestawienie wyników

#### Porównanie MSE najlepszych modeli każdego rozmiaru

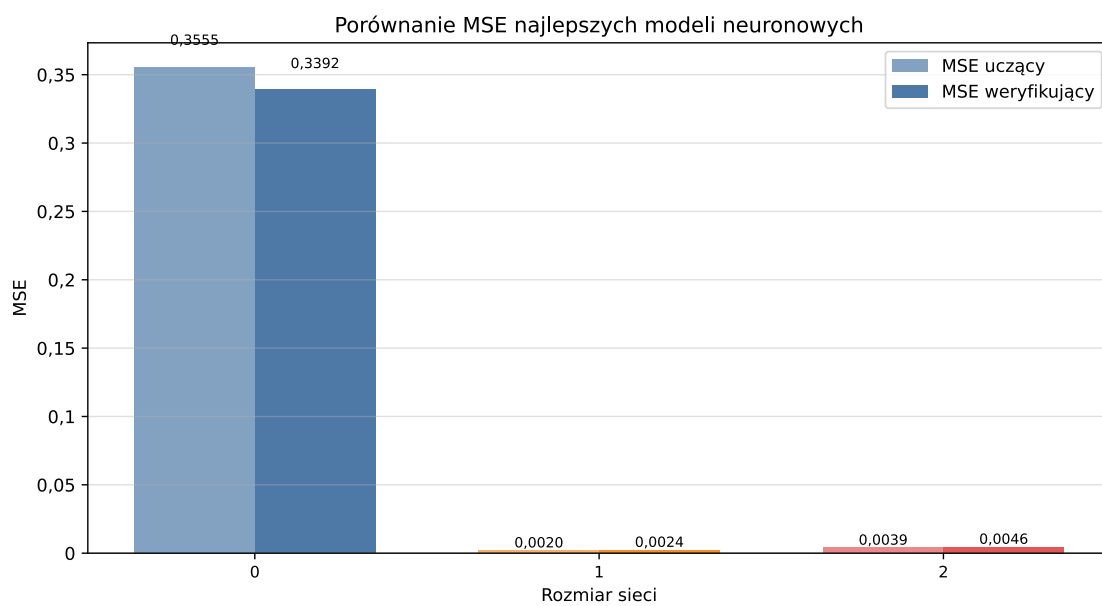


Tabela dostępna po uruchomieniu z `RUN_NEURAL = True`.

## 3e. Omówienie wyników

### Sieć zbyt mała (1 neuron, 4 par.):

Jeden neuron z  $\tanh$  może odwzorować jedynie monotoniczne krzywą sigmoidalną. Jeżeli charakterystyka statyczna jest bardziej złożona, sieć nie jest w stanie dobrze dopasować danych – MSE weryfikacyjny pozostaje wysoki dla wszystkich optymalizatorów, współczynników uczenia i punktów startowych. Wynik jest mało wrażliwy na wybór algorytmu i punktu startowego, bo obszar rozwiązań jest bardzo ubogi.

### Sieć umiarkowana (10 neuronów, 31 par.):

Sieć ma wystarczającą pojemność do aproksymacji nielinearności. Kluczowe obserwacje:

- **Wpływ optymalizatora:** Adam i RMSprop zazwyczaj schodzą szybciej niż SGD; przy tym samym  $\text{lr}$  SGD wymaga więcej epok lub wyższego  $\text{lr}$ .
- **Wpływ  $\text{lr}$ :** Zbyt małe  $\text{lr}$  (0,001) prowadzi do wolnej zbieżności w 4000 epokach. Zbyt duże  $\text{lr}$  (0,1) może powodować oscylacje lub zatrzymanie w lokalnym minimum. Optymalny zakres to  $\text{lr} \approx 0,01$ .
- **Wpływ punktu startowego:** Przy umiarkowanej sieci dobry optymalizator i odpowiedni  $\text{lr}$  prowadzą do podobnych wyników niezależnie od startu (stabilna zbieżność), choć mogą wystąpić lokalne minima.

### Sieć zbyt duża (500 neuronów, 1 501 par.):

Liczba parametrów przekracza 12-krotnie liczbę danych uczących. Sieć może **zapamiętać** dane zamiast nauczyć się ich wzorca:

- MSE uczący jest bardzo niski (bliski zera przy wystarczającej liczbie epok),
- MSE weryfikujący może być wyraźnie wyższy niż dla sieci umiarkowanej – to **przeuczenie** (*overfitting*).
- Różne punkty startowe prowadzą do bardziej rozproszonych wyników, bo przestrzeń wag jest ogromna.

### Ogólne wnioski:

1. Dla tego zadania (statyczna identyfikacja) sieć z 10 neuronami jest rozsądnym wyborem.
2. Algorytm Adam z  $\text{lr} \approx 0,01$  zapewnia najszybszą i najbardziej niezawodną zbieżność.
3. Modele neuronowe osiągają niższe MSE weryfikacyjne niż model liniowy, zbliżone do modelu wielomianowego wyższego stopnia, ale bez ryzyka "wibracji" na krańcach przedziału.
4. Porównując z NARX: statyczny model neuronowy aproksymuje tylko charakterystykę  $y(u)$ , natomiast NARX modeluje pełną dynamikę – cel jest inny.